



Gustavo Teodoro Bauke

Interface Web para Regressão Simbólica

Santo André - SP

2025

Gustavo Teodoro Bauke

Interface Web para Regressão Simbólica

Universidade Federal do ABC
Curso de Ciência da Computação

Orientador: Prof. Dr. Fabrício Olivetti de França

Santo André - SP
2025

Dedico este trabalho a todos meus amigos e familiares, que me apoiaram e me encorajaram a sempre seguir meus sonhos, mesmo quando eles pareciam impossíveis. Sem o amor e o suporte de vocês, eu não teria chegado onde estou hoje. Obrigado por acreditarem em mim e por me inspirarem a ser a melhor versão de mim mesmo. Este trabalho é para vocês!

AGRADECIMENTOS

Agradeço ao meu orientador, Prof. Dr. Fabrício Olivetti de França, por sua orientação e apoio durante todo o processo de desenvolvimento deste trabalho. Ao Movimento Empresa Júnior, por me mostrar que sonhar grande é possível e por me proporcionar experiências que levarei comigo para toda a vida. Ao Time do Penta e à NSPhamília, por estarem sempre dispostos a me ajudar e por me inspirarem a ser um ser humano melhor. E, por fim, à UFABC, por me mostrar que a interdisciplinaridade é fundamental para a formação de um ser humano completo e por me proporcionar uma educação de qualidade.

"Computadores são incrivelmente rápidos, precisos e estúpidos; humanos são incrivelmente lentos, imprecisos e brilhantes; juntos eles são poderosos além da imaginação." (Albert Einstein)

RESUMO

O uso de técnicas de inteligência artificial tem se tornado cada vez mais comum em diversas áreas, contudo, métodos tradicionais como redes neurais frequentemente atuam como "caixas-pretas", dificultando a compreensão lógica de seus resultados. Nesse cenário, a regressão simbólica ganha destaque ao buscar expressões matemáticas interpretáveis que descrevem a relação entre variáveis em um conjunto de dados. Apesar de sua relevância, ferramentas atuais de regressão simbólica (como TuringBot e HeuristicLab) apresentam limitações de acessibilidade e exigem execução local, o que pode ser um obstáculo para máquinas com baixo poder computacional, além de não permitirem a criação de pipelines completos de dados. Para superar esses desafios, este trabalho propõe o Spectrum, uma nova plataforma web para exploração e manipulação de modelos de regressão simbólica com processamento e treinamento remotos. Desenvolvida sob os princípios do Domain-Driven Design (DDD) em uma arquitetura cliente-servidor, a solução utiliza workers assíncronos para executar algoritmos de ponta como o Eggp, que otimiza as buscas utilizando grafos de igualdade (e-graphs). O grande diferencial da plataforma é a introdução da Inference Query Language (IQL), uma linguagem de domínio com sintaxe similar ao SQL. A IQL permite que os usuários realizem consultas condicionais declarativas sobre o vasto espaço de soluções matemáticas encontradas, filtrando as expressões por complexidade, erro ou padrões estruturais específicos. Conclui-se que a plataforma proposta oferece um ambiente integrado moderno, que democratiza o acesso e unifica os processos de exploração, treinamento e documentação em regressão simbólica.

Palavras-chave: Regressão Simbólica. Regressão. Aprendizado de Máquina. Interface Web. Grafos de Igualdade. Linguagem de Consulta.

ABSTRACT

The use of artificial intelligence techniques has become increasingly common in various fields, however, traditional methods such as neural networks often act as "black boxes", making it difficult to understand the logical reasoning behind their results. In this scenario, symbolic regression stands out by seeking interpretable mathematical expressions that describe the relationship between variables in a dataset. Despite its relevance, current symbolic regression tools (such as TuringBot and HeuristicLab) present accessibility limitations and require local execution, which can be an obstacle for machines with low computational power, in addition to not allowing the creation of complete data pipelines. To overcome these challenges, this work proposes Spectrum, a new web platform for exploring and manipulating symbolic regression models with remote processing and training. Developed under the principles of Domain-Driven Design (DDD) in a client-server architecture, the solution uses asynchronous workers to run state-of-the-art algorithms like Eggp, which optimizes searches using equality graphs (e-graphs). The great differential of the platform is the introduction of the Inference Query Language (IQL), a domain-specific language with SQL-like syntax. IQL allows users to perform declarative conditional queries on the vast space of mathematical solutions found, filtering expressions by complexity, error, or specific structural patterns. It is concluded that the proposed platform offers a modern integrated environment that democratizes access and unifies the processes of exploration, training, and documentation in symbolic regression.

Keywords: Symbolic Regression. Regression. Machine Learning. Web Interface. Equality Graphs. Query Language.

SUMÁRIO

	Lista de ilustrações	10
	Lista de Códigos	12
1	INTRODUÇÃO	13
1.1	Contextualização	13
1.2	Objetivos	13
1.3	Justificativa	14
1.4	Organização deste trabalho	14
2	REVISÃO DA LITERATURA	16
2.1	Algoritmos de Regressão	16
2.2	Algoritmos Genéticos	16
2.2.1	Operador de seleção	17
2.2.2	Operador de crossover	17
2.2.3	Operador de mutação	17
2.3	Regressão Simbólica	18
2.3.1	Programação Genética para Regressão Simbólica	18
2.3.2	Saturação de igualdade para Regressão Simbólica	20
2.3.2.1	Grafos de Igualdade	21
2.3.2.2	Saturação de igualdade	22
2.3.2.3	Algoritmo: Eggp	22
2.3.3	Outros métodos para Regressão Simbólica	23
2.3.3.1	Interação-Transformação	23
2.3.3.2	Regressão Simbólica Exaustiva	25
2.4	Ferramentas para Regressão Simbólica	25
2.5	Conclusão	26
3	MATERIAIS E MÉTODOS	27
3.1	Metodologia	27
3.2	Ambiente de Desenvolvimento e Ferramentas	27
3.2.1	Desenvolvimento Front-end	27
3.2.2	Estrutura do Projeto	29
3.3	Arquitetura	31
3.3.1	Visão Geral e Fluxo de Dados	32
3.3.1.1	Front-End	32

3.3.1.2	Back-End	33
3.3.1.3	Workers	34
3.3.1.4	Armazenamento	35
3.3.2	Abstrações e Conceitos Base	37
3.3.2.1	Abstrações de Banco de Dados	38
3.3.3	Abstrações de Domínio	41
4	PLATAFORMA WEB PARA EXPLORAÇÃO DE ALGORITMOS DE REGRESSÃO SIMBÓLICA	48
4.1	Panorama Geral	48
4.2	Histórias de Usuário	48
4.3	Interface Web e Blocos de Visualização	49
4.4	Requisitos não-funcionais	50
4.5	Back-End	50
4.5.1	Usuários	51
4.5.2	Datasets	54
4.5.3	Profiles	55
4.5.3.1	Jobs	57
4.5.3.2	Runs	60
4.5.3.3	Models	61
4.5.3.4	Blocks, InferenceRuns, InferenceResults	62
4.6	Workers	66
4.6.1	Treinamento	66
4.6.2	Validação	67
4.6.3	Consultas IQL	68
4.7	Front-End	69
4.7.1	Tela de Login	69
4.7.2	Tela de Cadastro	70
4.7.3	Editor	70
4.7.3.1	Navegação	71
4.7.3.2	Upload de Datasets	73
4.7.3.3	Aba de Profiles	75
5	LINGUAGEM DE DOMÍNIO PARA EXPLORAÇÃO DE REGRES- SÃO SIMBÓLICA	80
5.1	Contexto	80
5.2	Inference Query Language	80
5.2.1	Gramática Formal	81
5.2.2	Exemplos de Consultas	85
5.3	Execução	85

5.3.1	Análise Léxica (Tokenização)	86
5.3.2	Análise Sintática e o Pratt Parser	87
5.3.3	Execução	87
6	CONCLUSÃO	89
6.1	Limitações e Trabalhos Futuros	89
6.2	Conclusão	90
	Referências	91

LISTA DE ILUSTRAÇÕES

Figura 2 – Exemplo de uma árvore de sintaxe abstrata (AST) para a expressão $\log(x/2) + 3x$	18
Figura 3 – Exemplo de cruzamento entre duas expressões representadas por árvores de sintaxe abstrata (ASTs).	20
Figura 4 – Exemplo de mutação em uma expressão representada por uma árvore de sintaxe abstrata (AST).	20
Figura 5 – Exemplo de um grafo de igualdade, adaptado de (FRANÇA; KRONBERGER, 2025a).	21
Figura 6 – Exemplo de árvores geradas por regressão simbólica exaustiva.	25
Figura 7 – Diagrama de alto nível da arquitetura do projeto.	32
Figura 8 – Diagrama de alto nível da arquitetura do front-end.	33
Figura 9 – Diagrama de alto nível da arquitetura do back-end.	34
Figura 10 – Diagrama de entidades do sistema	51
Figura 11 – Tela de login da plataforma Spectrum	69
Figura 12 – Tela de cadastro da plataforma Spectrum	70
Figura 13 – Tela inicial do Editor da plataforma Spectrum	71
Figura 14 – Elementos de navegação da barra de atividades e da barra lateral do Editor da plataforma Spectrum	72
Figura 15 – Barra lateral do Editor da plataforma Spectrum com a visão de Datasets	72
Figura 16 – Tela de upload de datasets da plataforma Spectrum	73
Figura 17 – Menu de filtros avançados presente nas visões de Profiles da plataforma Spectrum	74
Figura 18 – Seção de configuração dos artefatos do dataset sendo criado na plataforma Spectrum	74
Figura 19 – Seção de pré-visualização dos dados presentes nos arquivos CSV enviados na plataforma Spectrum	75
Figura 20 – Tela de uma Profile na plataforma Spectrum, onde é possível observar as seções de metadados, datasets, Jobs e blocos	76
Figura 21 – Área de blocos de uma Profile na plataforma Spectrum, onde os usuários podem configurar e organizar os blocos de visualização de acordo com suas necessidades específicas	76
Figura 22 – Exemplo de resultados de uma consulta IQL sendo exibidos em tempo real por meio de Server Side Events (SSE) na plataforma Spectrum	77

Figura 23 – Seção de Jobs de uma Profile na plataforma Spectrum, onde os usuários podem configurar novos Jobs, visualizar Jobs configurados, executar Jobs, acompanhar o progresso de execução e analisar o histórico de execuções de cada Job	77
Figura 24 – Tela de configuração de um novo Job na plataforma Spectrum, onde os usuários podem configurar os meta-parâmetros do algoritmo Eggp, como o número de gerações, o tamanho da população e as funções a serem utilizadas, entre outros	78
Figura 25 – Seção de modelos de uma Profile na plataforma Spectrum, onde é possível visualizar os modelos de regressão simbólica gerados a partir de suas execuções, bem como acessar as métricas de performance associadas a cada modelo	78
Figura 26 – Comparação entre as expressões da fronteira de pareto encontradas durante o processo de validação na plataforma Spectrum	79
Figura 27 – Comparação entre os valores reais do conjunto de validação e os valores preditos por cada expressão na plataforma Spectrum	79
Figura 28 – Gramática formal da Inference Query Language (IQL) em notação EBNF, definindo a sintaxe completa para consultas declarativas sobre modelos de regressão simbólica baseados em e-graphs	82
Figura 29 – Pipeline de processamento, análise e execução de uma consulta IQL, desde a entrada textual oriunda da <i>request</i> do usuário até o armazenamento dos dados	86
Figura 30 – Visualização da Árvore de Sintaxe Abstrata (AST) do Pratt Parser . .	87

LISTA DE CÓDIGOS

3.1	Bibliotecas utilizadas no front-end e suas versões	28
3.2	Ferramentas utilizadas no back-end e suas versões	30
3.3	Definição da interface de armazenamento	35
3.4	Implementação da interface de armazenamento no back-end	36
3.5	Definição de uma entidade mutável em nível de banco de dados	38
3.6	Definição de uma entidade imutável em nível de banco de dados	39
3.7	Definição de uma entidade versionada em nível de banco de dados	40
3.8	Definição de uma entidade mutável em nível de domínio	41
3.9	Definição de uma entidade imutável em nível de domínio	42
3.10	Definição de uma entidade versionada em nível de domínio	42
3.11	Definição de um protocolo de repositório para entidades mutáveis	42
3.12	Definição de um protocolo de repositório para entidades imutáveis	43
3.13	Definição de um protocolo de repositório para entidades versionadas	44
3.14	Definição de um protocolo de repositório para operações em lote	44
3.15	Definição de um protocolo para o padrão de código Unit of Work	45
3.16	Definição de um protocolo para recursos transacionais	47
4.1	Código SQLAlchemy para criação da tabela de usuários	52
4.2	Classe de domínio que define um usuário	52
4.3	Classe de domínio que define um Dataset	54
4.4	Classe de domínio que define um Profile	55
4.5	Classe de domínio que define um Job	57
4.6	Lista de funções de perda disponíveis	59
4.7	Lista de não terminais disponíveis	59
4.8	Classe de domínio que define um Run	60
4.9	Classe de domínio que define um Model	62
4.10	Classe de domínio que define um Block	63
4.11	Classe de domínio que define um bloco de consulta em linguagem IQL	63
4.12	Classe de domínio que define um bloco de consulta em linguagem IQL	64
4.13	Classe de domínio que define uma InferenceRun	64
4.14	Classe de domínio que define uma InferenceResult	65

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

Com evoluções tecnológicas constantes, o uso de técnicas de inteligência artificial (IA), principalmente algoritmos de aprendizado de máquina (AM), tem se tornado cada vez mais comum em diversas áreas (MAKKE; CHAWLA, 2024). Atualmente, as principais técnicas de IA utilizadas são baseadas em redes neurais e transformadores, que são capazes de aprender padrões complexos a partir de grandes volumes de dados. No entanto, essas técnicas muitas vezes resultam em modelos que são considerados "caixas-pretas", ou seja, cuja lógica interna é difícil de interpretar e compreender totalmente (FRANÇA; KRONBERGER, 2025a). Em algumas áreas, como na medicina, a interpretabilidade dos modelos é crucial, pois permite um entendimento mais profundo dos fenômenos capturados pelos dados.

Dessa forma, outras técnicas de IA, como a regressão simbólica, têm ganhado destaque como forma de obter modelos mais interpretáveis. A regressão simbólica é uma técnica que busca encontrar expressões matemáticas que melhor se ajustam aos dados observados e que respeitem um conjunto pré-determinado de operações matemáticas (FRANÇA; KRONBERGER, 2025a,b; BARTLETT; DESMOND; FERREIRA, 2024). Algoritmos de regressão simbólica são capazes de gerar modelos matemáticos que podem ser facilmente interpretados e analisados, o que os torna uma alternativa interessante para aplicações onde a transparência é importante. Várias técnicas podem ser utilizadas para implementar a regressão simbólica, incluindo algoritmos evolutivos ou técnicas híbridas que combinam redes neurais com algoritmos evolutivos.

1.2 OBJETIVOS

O objetivo deste trabalho é propor uma nova plataforma para manipulação de modelos de regressão simbólica. Para isso, primeiramente será feita uma revisão de literatura, apresentando as principais características, técnicas e aplicações da regressão simbólica, bem como comparação com outras técnicas de inferência. Dentre os objetivos específicos da plataforma proposta, podemos destacar:

1. Gerenciamento de pipelines de manipulação de dados
2. Exploração das soluções encontradas
3. Identificação de padrões presentes nas soluções encontradas

4. Seleção dos meta-parâmetros do algoritmo
5. Plotagem de gráficos com informações pertinentes sobre o conjunto de dados
6. Manipulação do conjunto de dados
7. Geração de datasets aleatórios para testes da ferramenta
8. Consulta condicional de soluções por meio de uma linguagem similar ao SQL

1.3 JUSTIFICATIVA

Atualmente, ferramentas como TuringBot e HeuristicLab podem ser utilizadas para criação de modelos de regressão simbólica. Apesar das vastas funcionalidades dessas ferramentas, elas possuem limitações em termos de acessibilidade e praticidade de uso, exigindo, muitas vezes, instalação local. Para máquinas fracas, executar os algoritmos de regressão simbólica pode se tornar um desafio, necessitando alternativas de execução remota. Ademais, as ferramentas existentes não permitem a criação de pipelines completas de importação, processamento e extração de dados. A interface web proposta neste trabalho visa superar essas limitações, oferecendo uma plataforma acessível, na qual o treinamento dos modelos é feito remotamente, permitindo que usuários possam explorar a regressão simbólica de forma mais intuitiva.

Além disso, essa plataforma permitirá que usuários explorem soluções por meio de uma linguagem de consulta, similar ao SQL, para selecionar padrões matemáticos de interesse dentro do espaço de soluções proposto pelo algoritmo, garantindo que conhecimentos prévios sobre os fenômenos estudados possam ser incorporados ao processo de modelagem.

A plataforma também permitirá a criação de pipelines completos para manipulação de conjuntos de dados de forma totalmente autônoma, permitindo a construção de processos complexos que dependem de modelos de regressão simbólica, podendo ser utilizada para mecanismos de previsão por exemplo.

1.4 ORGANIZAÇÃO DESTE TRABALHO

Este trabalho está organizado da seguinte forma: o capítulo 2 traz uma revisão da literatura atual sobre regressão simbólica, além de explorar os conceitos necessários para entendimento das principais implementações de algoritmos de regressão simbólica. O capítulo começa com uma revisão dos principais conceitos relacionados a regressão e algoritmos genéticos, depois o capítulo explica como algoritmos de regressão simbólica funcionam. Em seguida, o capítulo 2 discorre sobre as principais implementações

de regressão simbólica atuais e, por fim, há uma breve comparação entre as principais ferramentas de regressão simbólica disponíveis atualmente. O capítulo 3 apresenta as metodologias utilizadas na construção desse projeto. Os próximos dois capítulos apresentam, respectivamente, a proposta de plataforma web para exploração de algoritmos de regressão simbólica e a linguagem de consulta proposta para essa exploração. O capítulo 6 apresenta os resultados obtidos com a implementação da plataforma e, por fim, o capítulo 7 traz as conclusões do trabalho e sugestões para trabalhos futuros.

2 REVISÃO DA LITERATURA

2.1 ALGORITMOS DE REGRESSÃO

Regressão é o processo de descobrir relações entre variáveis de entrada e de saída a partir de um conjunto de dados (STULP; SIGAUD, 2015). As técnicas de regressão podem ser divididas em duas categorias principais: regressão paramétrica e não paramétrica.

A regressão paramétrica fixa uma relação funcional entre as variáveis e seu objetivo é estimar os parâmetros dessa relação. Por outro lado, a regressão não paramétrica não assume uma forma funcional específica, permitindo maior flexibilidade na modelagem de dados complexos (ANGELIS; SOFOS; KARAKASIDIS, 2023).

Algoritmos de regressão são comumente utilizados para interpolação e extrapolação de dados. Interpolação é o processo de estimar valores dentro do espaço de dados utilizados para o treinamento do modelo de regressão, enquanto extrapolação é o processo de estimar valores fora desse espaço. Diferentes algoritmos possuem propriedades diferentes em relação a suas capacidades de interpolação e extrapolação (ANGELIS; SOFOS; KARAKASIDIS, 2023).

Existem diversas formas de implementar algoritmos de regressão, a depender do tipo de regressão desejado para o modelo. Este capítulo focará em algoritmos de regressão não paramétricos de ordem arbitrária, especificamente no algoritmo de Regressão Simbólica implementado via programação genética e grafos de igualdade. Além disso, serão abordadas comparações com outras técnicas de implementação de regressão simbólica.

2.2 ALGORITMOS GENÉTICOS

Algoritmos genéticos (AGs) são uma classe de algoritmos inspirados na evolução natural, onde soluções são evoluídas ao longo de gerações para resolver problemas complexos. A ideia principal do algoritmo é simular o processo de seleção natural, no qual a pressão exercida pelo ambiente seleciona os melhores indivíduos de uma espécie. Esses algoritmos utilizam operações como seleção, cruzamento e mutação para explorar o espaço de soluções (ANGELIS; SOFOS; KARAKASIDIS, 2023; KRONBERGER; BURLACU et al., 2024). O objetivo principal é otimizar um objetivo arbitrário em relação a um conjunto de variáveis de entrada (BEYER; SCHWEFEL, 2002). Algoritmos genéticos são amplamente utilizados em problemas de busca, otimização, decisão e aprendizado de máquina (LAMBORA; GUPTA; CHOPRA, 2019).

O funcionamento básico do algoritmo segue os seguintes passos:

1. Uma população inicial de soluções é gerada aleatoriamente;
2. A cada iteração, os indivíduos da população são avaliados com base em uma função de aptidão, que indica o quão bem cada indivíduo resolve o problema;
3. Os melhores indivíduos são selecionados para reprodução;
4. Operadores de cruzamento e mutação são aplicados para gerar novos indivíduos;
5. A nova população é formada com os indivíduos gerados;
6. O processo é repetido até que um critério de parada seja atingido.

Para implementação do algoritmo, é necessário definir a forma de representação dos indivíduos. Uma representação comumente utilizada é a codificação por cadeia de caracteres binária, na qual cada indivíduo é representada por uma sequência de bits, no qual cada bit representa uma propriedade do indivíduo (KATOCH; CHAUHAN; KUMAR, 2021). No algoritmo de regressão simbólica, abordado mais adiante, os indivíduos são representados por árvores de sintaxe abstrata, as quais codificam operações matemáticas.

2.2.1 OPERADOR DE SELEÇÃO

Seleção é o processo de escolher quais indivíduos da população atual serão utilizados para reprodução. Diversas estratégias de seleção podem ser utilizadas, como seleção por torneio e roleta. A seleção por torneio envolve a escolha dos melhores indivíduos de um subconjunto aleatório da população baseando-se nos maiores valores da função de aptidão. A seleção por roleta envolve a escolha de indivíduos com base em uma distribuição probabilística, onde indivíduos com maior aptidão têm maior probabilidade de serem selecionados (KATOCH; CHAUHAN; KUMAR, 2021).

2.2.2 OPERADOR DE CROSSOVER

Cruzamento é o processo de combinação dos genomas de dois indivíduos para geração de novos indivíduos (LAMBORA; GUPTA; CHOPRA, 2019; KATOCH; CHAUHAN; KUMAR, 2021). Normalmente, o cruzamento é realizado escolhendo-se um ponto aleatório de corte na representação genética dos indivíduos e trocando-se as partes dos genomas a partir desse ponto.

2.2.3 OPERADOR DE MUTAÇÃO

Mutação é o processo de alteração aleatória de um ou mais genes de um indivíduo para introduzir diversidade na população. O objetivo da mutação é evitar a convergência

prematura para solução sub-ótimas (LAMBORA; GUPTA; CHOPRA, 2019). Assim como no cruzamento, a mutação depende da forma utilizada para codificação dos indivíduos. Em uma representação em que os genes são expressos por uma sequência de bits, a mutação pode ser realizada invertendo-se um ou mais bits aleatoriamente.

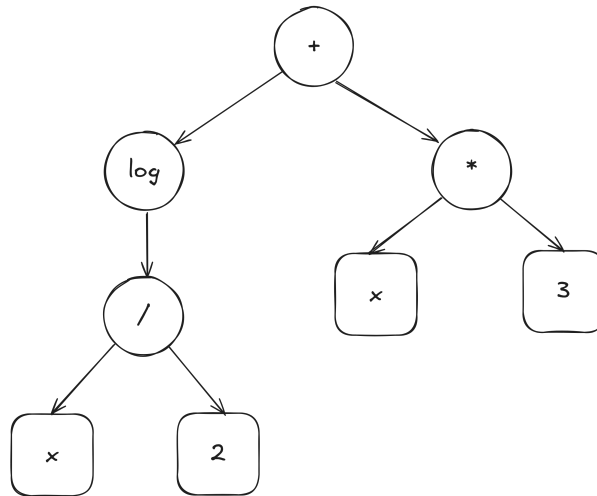
2.3 REGRESSÃO SIMBÓLICA

Regressão simbólica é uma técnica que busca encontrar expressões matemáticas que melhor descrevem a relação entre variáveis de entrada e saída em um conjunto de dados e pode ser categorizada como uma forma de regressão não paramétrica (FRANÇA; KRONBERGER, 2025a,b). Algoritmos considerados *State of the Art* para regressão simbólica utilizam algoritmos genéticos para evoluir expressões matemáticas que representam essas relações (OLIVETTI DE FRANÇA, 2018; KRONBERGER; BURLACU et al., 2024; FRANÇA; KRONBERGER, 2025a). No entanto, a regressão simbólica também pode ser implementada usando outros métodos, como redes neurais e transformadores (ANGELIS; SOFOS; KARAKASIDIS, 2023).

2.3.1 PROGRAMAÇÃO GENÉTICA PARA REGRESSÃO SIMBÓLICA

A programação genética utilizada na regressão simbólica evolui expressões matemáticas representadas por árvores de sintaxe abstrata (ASTs). Cada nó da árvore representa uma operação matemática (adição, subtração, multiplicação, exponenciação, etc.) e as folhas representam variáveis de entrada ou constantes. A Figura 2 ilustra um exemplo de árvore de sintaxe abstrata para a expressão $\log(x/2) + 3x$.

Figura 2 – Exemplo de uma árvore de sintaxe abstrata (AST) para a expressão $\log(x/2) + 3x$.



Nós que representam operações matemáticas são representados por círculos, enquanto nós que representam variáveis de entrada ou constantes são representados por

quadrados. Nós internos da árvore são chamados de não terminais, enquanto os nós folhas são chamados de terminais. O processo de busca envolve encontrar a melhor combinação de nós terminais e não terminais para minimizar a diferença entre os valores previstos pela expressão e os valores reais do conjunto de dados respeitando as restrições de complexidade e interpretabilidade da expressão (ANGELIS; SOFOS; KARAKASIDIS, 2023).

O processo de evolução se inicia com uma população inicial de expressões aleatórias. Em cada iteração, as expressões são avaliadas com base em uma função de aptidão, que mede o quão bem cada expressão se ajusta aos dados. As melhores expressões são selecionadas para reprodução, onde passam por operações de cruzamento e mutação para gerar novas expressões que são adicionadas à nova população. Esse processo continua até que um critério de parada seja atingido. Critérios de parada comuns incluem um número máximo de gerações, obtenção da expressão ideal ou convergência da população. Comumente, uma combinação entre todos os critérios é utilizada para garantir que o processo de evolução não seja interrompido prematuramente (ANGELIS; SOFOS; KARAKASIDIS, 2023; KRONBERGER; BURLACU et al., 2024).

A Figura 3 ilustra o processo de cruzamento entre duas expressões. Na parte superior, os indivíduos pais e, na parte inferior, os indivíduos produzidos por eles. Durante esse processo, subárvores aleatórias são selecionadas em cada expressão e trocadas entre si para criar dois novos indivíduos. Note que o cruzamento pode resultar em expressões mais complexas que os pais originais, o que pode ser controlado por meio de restrições de profundidade ou tamanho da árvore.

A Figura 4 ilustra o processo de mutação, onde uma subárvore aleatória é selecionada e substituída por uma nova subárvore gerada aleatoriamente. A mutação introduz diversidade na população, ajudando a evitar a convergência prematura para soluções sub-ótimas. Na esquerda, o indivíduo original, e na direita, o indivíduo após a mutação.

É importante destacar que esse algoritmo não precisa necessariamente retornar uma única expressão. Um conjunto de expressões pode ser retornado, permitindo ao usuário escolher a que melhor se adequa às suas necessidades (ANGELIS; SOFOS; KARAKASIDIS, 2023).

Apesar dos avanços recentes, a regressão simbólica ainda enfrenta vários desafios. A complexidade computacional decorrente do grande espaço de busca de expressões possíveis pode tornar o processo de evolução lento e ineficiente. Além disso, a interpretabilidade das expressões geradas pode ser comprometida quando elas se tornam excessivamente complexas, dificultando a compreensão por parte dos usuários finais (ANGELIS; SOFOS; KARAKASIDIS, 2023; KRONBERGER; BURLACU et al., 2024).

Além disso, a codificação de expressões como árvores pode levar o algoritmo de regressão a visitar expressões logicamente equivalentes várias vezes (FRANÇA; KRON-

Figura 3 – Exemplo de cruzamento entre duas expressões representadas por árvores de sintaxe abstrata (ASTs).

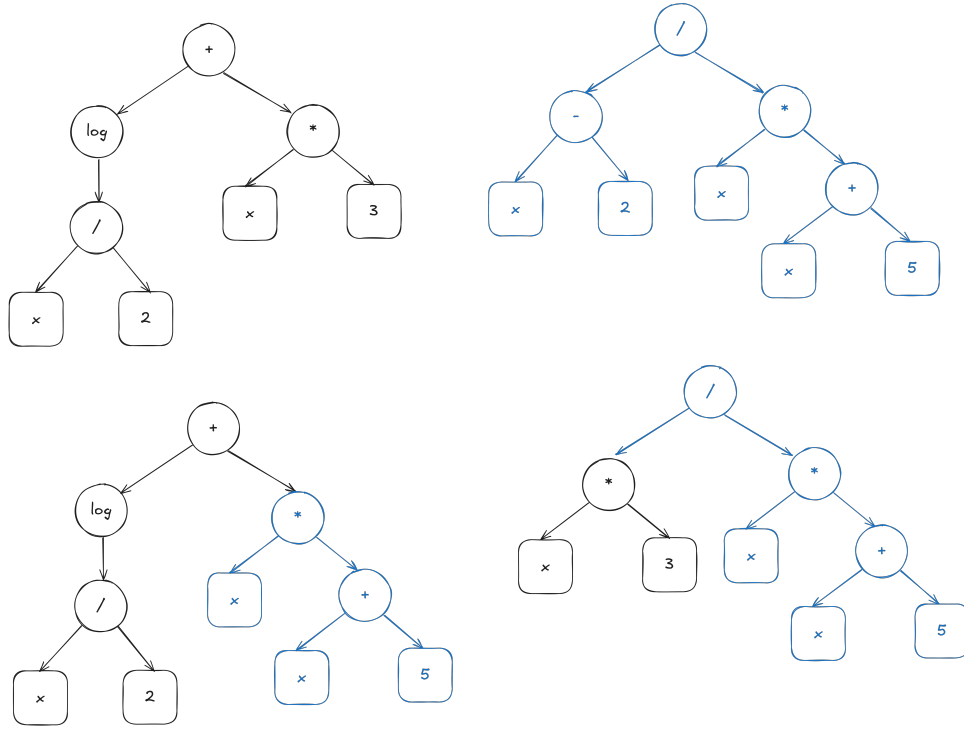
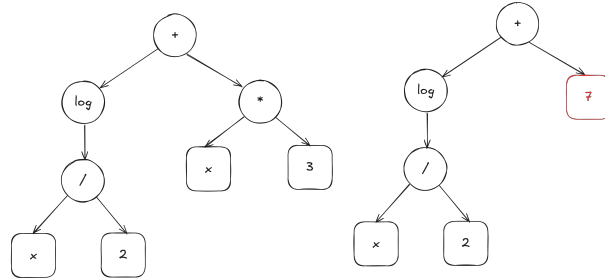


Figura 4 – Exemplo de mutação em uma expressão representada por uma árvore de sintaxe abstrata (AST).

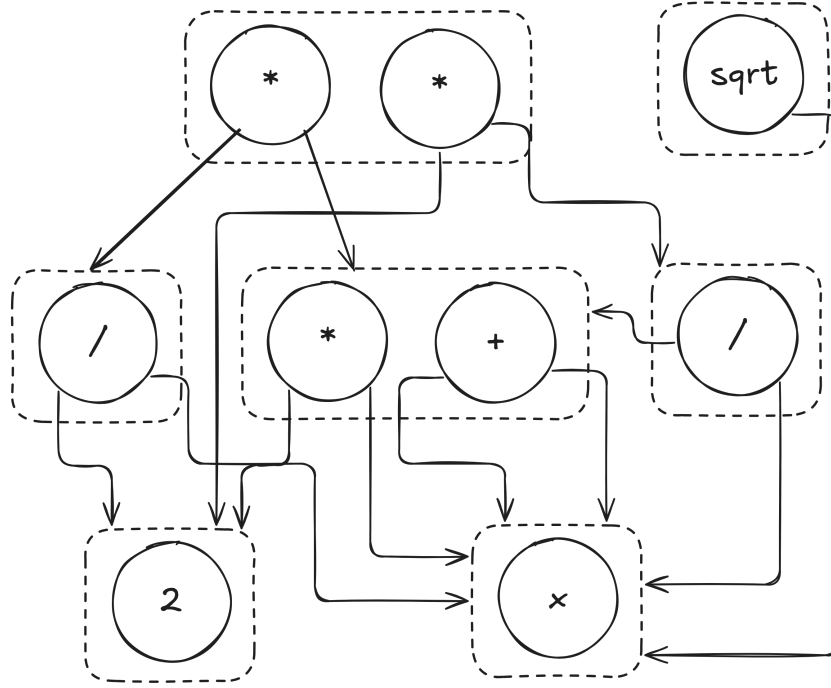


BERGER, 2025a). Por exemplo, as expressões $x + y$ e $y + x$ são logicamente iguais, mas seriam representadas por árvores diferentes. Testes empíricos indicam que apenas 40% das expressões geradas por algoritmos genéticos para regressão simbólica são únicas (KRONBERGER; OLIVETTI DE FRANÇA et al., 2024).

2.3.2 SATURAÇÃO DE IGUALDADE PARA REGRESSÃO SIMBÓLICA

Diante dos desafios enfrentados pela regressão simbólica tradicional, uma abordagem alternativa que tem ganhado destaque é o uso de grafos de igualdade (Equality Graphs - e-graphs) para representar e manipular expressões matemáticas (FRANÇA; KRONBERGER, 2025a,b). E-graphs são estruturas de dados que armazenam múltiplas expressões equivalentes de forma compacta, permitindo a aplicação eficiente de transformações algébricas e simplificações nos indivíduos presentes no espaço de busca. Utilizando essa

Figura 5 – Exemplo de um grafo de igualdade, adaptado de (FRANÇA; KRONBERGER, 2025a).



estrutura de dados, é possível realizar uma operação chamada de saturação de igualdade, que consiste em aplicar regras de reescrita para gerar todas as expressões logicamente equivalentes a uma dada expressão inicial.

2.3.2.1 GRAFOS DE IGUALDADE

Grafos de igualdade são estruturas de dados que representam expressões matemáticas e suas equivalências de maneira eficiente (FRANÇA; KRONBERGER, 2025a,b). Elementos matemáticos (operadores, variáveis e constantes) são representadas por e-nodes, enquanto classes de expressões equivalentes, ou seja, conjuntos de e-nodes equivalentes, são representadas por e-classes. Cada e-class contém um conjunto de e-nodes que representam diferentes formas de expressar a mesma expressão matemática. Cada e-node aponta para e-classes filhas, que representam os operandos da operação matemática representada pelo e-node, permitindo que variações de uma mesma expressão compartilhem subárvores e sejam representadas sem consumo elevado de memória.

A Figura 5 ilustra um grafo de igualdade. Note que as expressões $2x$ e $x + x$ pertencem à mesma e-class, uma vez que são logicamente equivalentes. A implementação de e-graphs utiliza uma estrutura union-find para gerenciar a união de e-classes quando novas equivalências são descobertas durante o processo de saturação (FRANÇA; KRONBERGER, 2025a). Uma e-class armazena uma lista de e-nodes que pertencem a ela, uma lista de e-nodes pais (e-nodes que possuem essa e-class como filha) e demais informações auxiliares para otimizar operações de busca e união.

2.3.2.2 SATURAÇÃO DE IGUALDADE

Saturação de igualdade é o processo de aplicar regras de reescrita a uma árvore para gerar expressões logicamente equivalentes. Quando uma equivalência entre expressões é encontrada, as e-classes correspondentes são fundidas, sinalizando que diferentes caminhos estruturais possuem mesmo significado semântico. O processo é eficiente porque todas as regras de reescrita são aplicadas paralelamente (FRANÇA; KRONBERGER, 2025a), utilizando a estrutura de e-graphs para evitar a duplicação de expressões. O processo de saturação acontece em quatro etapas:

1. Elenque todas as regras de equivalência para o estado atual do e-graph;
2. Aplique todas as regras;
3. Junte e-classes equivalentes;
4. Repita os passos anteriores até que nenhuma nova equivalência seja encontrada.

Após a aplicação repetida de todas as regras, o e-graph é considerado saturado. Nesse estado, todas as expressões equivalentes possíveis foram geradas e armazenadas no e-graph.

2.3.2.3 ALGORITMO: EGGP

Eggp (E-graph Genetic Programming) é uma implementação de um algoritmo genético para regressão simbólica que utiliza e-graphs e saturação de igualdade (FRANÇA; KRONBERGER, 2025a). O algoritmo segue os passos tradicionais de um algoritmo genético, mas com algumas diferenças importantes:

1. A cada nova expressão inserida no e-graph, é executada uma etapa de saturação, agrupando todas as expressões logicamente equivalentes;
2. Os operadores de mutação e cruzamento são adaptados para geração de expressões não visitadas;

Os operadores de mutação e cruzamento utilizam das informações do e-graph para gerar expressões ainda não visitadas. No cruzamento, uma subárvore aleatória de um dos pais é substituída por uma subárvore aleatória escolhida de um conjunto de todas as possíveis subárvores do outro pai. O conjunto é construído de tal forma que todos seus elementos, ao substituírem a subárvore original, geram expressões novas.

Na mutação, o processo tradicional é seguido, substituindo-se uma subárvore aleatória por uma nova subárvore gerada aleatoriamente. No entanto, caso a nova expressão já exista no e-graph, a subárvore é substituída por outra aleatória que faça parte do

conjunto de símbolos de mesma aridade da subárvore original. Da mesma forma que no cruzamento, esse conjunto é formado de tal maneira que todas as substituições geram expressões novas.

Dessa forma, a utilização de grafos de igualdade reduz drasticamente o espaço de busca da regressão simbólica, permitindo maior eficiência do algoritmo sem aumento significativo no custo computacional. Durante o processo de evolução, é comum que expressões semanticamente equivalentes sejam geradas e exploradas (FRANÇA; KRONBERGER, 2025a; KRONBERGER; OLIVETTI DE FRANÇA et al., 2024). A utilização de e-graphs permite que esses caminhos repetidos dentro do espaço de soluções sejam podados do processo de busca, otimizando o algoritmo.

2.3.3 OUTROS MÉTODOS PARA REGRESSÃO SIMBÓLICA

Apesar de algoritmos genéticos serem os mais comuns para implementação de regressão simbólica, outros métodos podem ser utilizados (BARTLETT; DESMOND; FERREIRA, 2024; MAKKE; CHAWLA, 2024):

1. Aprendizado por reforço;
2. Redes neurais;
3. Transformadores.

Para fins de comparação, esse trabalho também aborda uma implementação de regressão simbólica conhecida como Interação-Transformação (IMAI ALDEIA; FRANÇA, 2018), que utiliza uma abordagem diferente para geração de expressões matemáticas. Além disso, também será abordada uma técnica conhecida como Regressão Simbólica Exaustiva (BARTLETT; DESMOND; FERREIRA, 2024), que utiliza uma técnica similar à Interação-Transformação, mas que visa explorar todo o espaço de expressões possíveis dentro de um recorte delimitado, garantindo que a solução ótima seja encontrada.

2.3.3.1 INTERAÇÃO-TRANSFORMAÇÃO

A regressão simbólica utilizando da estrutura de Interação-Transformação (IT) é uma abordagem que restringe o espaço de busca de expressões matemáticas para aquelas que podem ser representadas como uma combinação polinomial das variáveis seguidas por uma transformação não-linear (IMAI ALDEIA; FRANÇA, 2018). Diferentemente dos algoritmos genéticos, nessa implementação, algumas relações não podem ser representadas.

Nessa representação, os termos são representados por uma tupla $(\mathbf{P}, \mathbf{t})$, onde $\mathbf{P}$ é um vetor de expoentes e $\mathbf{t}$ é uma função de transformação sendo aplicada sobre os termos do polinômio (IMAI ALDEIA; FRANÇA, 2018). Como um exemplo dessa representação,

considere uma regressão de quatro variáveis de entrada, x_1 , x_2 , x_3 e x_4 . Considere que a função de transformação seja $t(x) = \log(x)$. Então, teríamos a seguinte representação:

$$\begin{aligned}\mathbf{T} &= (x_1, x_2, x_3, x_4) \\ \mathbf{t}_1 &= ([1, 0, 0, 0], \log) \\ \mathbf{t}_2 &= ([0, 1, 0, 0], \log) \\ \mathbf{t}_3 &= ([0, 0, 1, 0], \log) \\ \mathbf{t}_4 &= ([0, 0, 0, 1], \log)\end{aligned}\tag{2.1}$$

Para gerar novas expressões, o algoritmo utiliza uma abordagem de busca em largura. A cada iteração, o algoritmo gera todos os termos possíveis decorrentes da combinação dos termos já existentes de forma que $t_k = (P_i + P_j, id)$ e $t_l = (P_i - P_j, id)$. No exemplo anterior, teríamos para a combinação de t_1 e t_2 os seguintes termos:

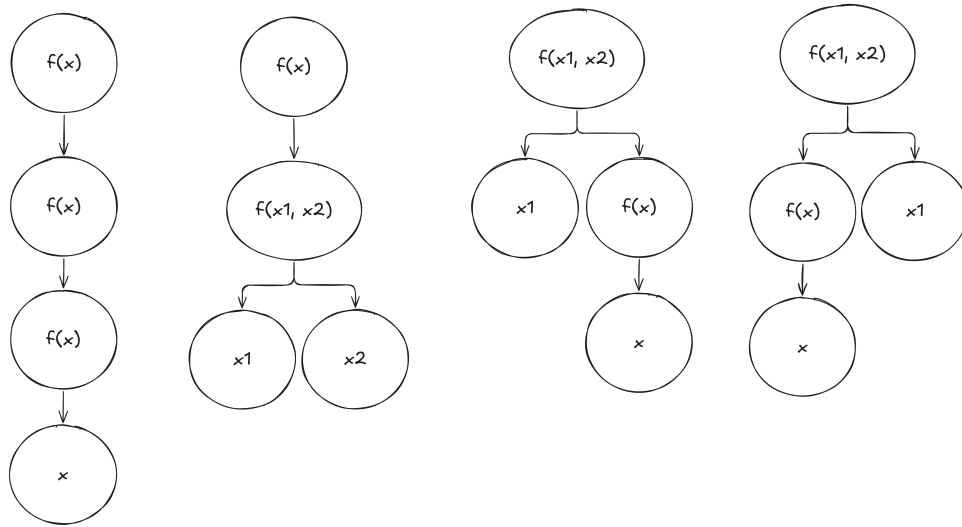
$$\begin{aligned}\mathbf{t}_5 &= ([1, 1, 0, 0], \log) \\ \mathbf{t}_6 &= ([1, -1, 0, 0], \log)\end{aligned}\tag{2.2}$$

Para etapa de transformação, para cada termo $(\mathbf{P}, t)$, o algoritmo gera um novo termo $(\mathbf{P}, t')$ para cada função de transformação $t'! = t$ disponível. No exemplo anterior, considerando que as funções de transformação disponíveis sejam $\log$, $\sqrt{x}$ e e^x , teríamos os seguintes termos:

$$\begin{aligned}\mathbf{t}_7 &= ([1, 0, 0, 0], \sqrt{x}) \\ \mathbf{t}_8 &= ([1, 0, 0, 0], e^x) \\ \mathbf{t}_9 &= ([0, 1, 0, 0], \sqrt{x}) \\ \mathbf{t}_{10} &= ([0, 1, 0, 0], e^x) \\ \mathbf{t}_{11} &= ([0, 0, 1, 0], \sqrt{x}) \\ \mathbf{t}_{12} &= ([0, 0, 1, 0], e^x) \\ \mathbf{t}_{13} &= ([0, 0, 0, 1], \sqrt{x}) \\ \mathbf{t}_{14} &= ([0, 0, 0, 1], e^x) \\ \mathbf{t}_{15} &= ([1, 1, 0, 0], \sqrt{x}) \\ \mathbf{t}_{16} &= ([1, 1, 0, 0], e^x) \\ \mathbf{t}_{17} &= ([1, -1, 0, 0], \sqrt{x}) \\ \mathbf{t}_{18} &= ([1, -1, 0, 0], e^x)\end{aligned}\tag{2.3}$$

Para cada expressão gerada, o algoritmo avalia a aptidão da expressão. Todas as expressões que não atingirem um valor mínimo de aptidão são descartadas (IMAI ALDEIA; FRANÇA, 2018).

Figura 6 – Exemplo de árvores geradas por regressão simbólica exaustiva.



2.3.3.2 REGRESSÃO SIMBÓLICA EXAUSTIVA

Semelhantemente a regressão simbólica utilizando Interação-Transformação, a regressão simbólica exaustiva busca limitar o espaço de busca, mas com o objetivo de explorar todo o espaço, garantindo que a solução ótima seja encontrada (BARTLETT; DESMOND; FERREIRA, 2024).

Nessa abordagem, a geração de expressões é feita por meio da geração de todas as combinações possíveis entre operadores de aridade zero, um e dois. A aridade zero é representada por constantes, a aridade um por funções unárias (como $\log$ e $\sqrt{}$) e a aridade dois por funções binárias (como $+$, $-$, $*$ e $/$) (BARTLETT; DESMOND; FERREIRA, 2024). Considere uma árvore de profundidade máxima $d = 4$ e um conjunto de operadores de aridade zero (x), um ($f(x)$) e dois ($f(x_1, x_2)$). Nesse caso, temos apenas quatro árvores válidas, conforme ilustrado na Figura 6.

Após a geração de todas as expressões, o algoritmo checa por expressões logicamente equivalentes, removendo-as do conjunto de expressões (BARTLETT; DESMOND; FERREIRA, 2024).

2.4 FERRAMENTAS PARA REGRESSÃO SIMBÓLICA

Existem diversas ferramentas disponíveis para exploração de algoritmos de regressão simbólica. Essas ferramentas variam desde implementações comerciais até bibliotecas de código aberto, cada uma com suas próprias características e funcionalidades.

HeuristicLab é uma ferramenta gráfica que, dentre outros algoritmos, suporta a exploração de algoritmos de regressão simbólica. Ele permite a criação de algoritmos personalizados e a visualização dos resultados de forma interativa. A ferramenta é extensível

e pode ser adaptada para diferentes necessidades de pesquisa (WAGNER et al., 2014). A ferramenta exibe a fronteira de Pareto final, detalhes estatísticos do modelo gerado, gráficos de desempenho e a representação final da árvore de busca.

Assim como a ferramenta anterior, TuringBot é uma interface gráfica para exploração de algoritmos de regressão simbólica. Ele mostra ao usuário as melhores expressões encontradas (fronteira de Pareto) para o conjunto de dados fornecido com algumas métricas simples do modelo criado (TURINGBOT SOFTWARE, 2020).

Para além de ferramentas comerciais, existem diversas bibliotecas que permitem a exploração de algoritmos de regressão simbólica. No caso do algoritmo Eggp, apresentado anteriormente, é possível utilizar a ferramenta *rEGGression* (FRANÇA; KRONBERGER, 2025b) para explorar os modelos gerados por esse algoritmo. Essa ferramenta permite que as expressões encontradas pelo modelo sejam consultadas por tamanho, complexidade e número de parâmetros numéricos; permite a inserção de novas expressões; permite a listagem de sub-expressões; e permite o uso de padrões para busca de expressões (FRANÇA; KRONBERGER, 2025b).

Apesar da existência de ferramentas que permitem exploração de modelos de regressão simbólica, poucas ferramentas permitem a criação de *pipelines* de dados para criação de modelos preditivos. As ferramentas disponíveis também não permitem a exploração de padrões dentro das expressões geradas.

2.5 CONCLUSÃO

A revisão de literatura apresentada neste capítulo retoma os principais conceitos e técnicas relacionados à regressão simbólica, incluindo algoritmos genéticos, programação genética e o uso de grafos de igualdade, dentre outras técnicas. Além disso, foram apresentadas ferramentas e implementações que permitem a exploração desses conceitos na prática. Nos próximos capítulos, será apresentada uma proposta de ambiente de exploração de algoritmos de regressão simbólica, com foco na implementação do algoritmo Eggp e na utilização de grafos de igualdade para otimização do processo de evolução de expressões matemáticas.

3 MATERIAIS E MÉTODOS

3.1 METODOLOGIA

O desenvolvimento deste projeto seguiu uma abordagem iterativa e incremental, baseando-se em metodologias ágeis em oposição ao modelo tradicional em cascata (*waterfall*). Enquanto o modelo cascata prevê uma sequência rígida de fases (requisitos, projeto, implementação, testes e manutenção), as metodologias ágeis permitem a adaptação contínua e a entrega frequente de valor (SOMMERVILLE, 2011).

Dentre as práticas ágeis, foram utilizados princípios do *Scrum*, como a divisão do trabalho em ciclos curtos e a revisão constante dos requisitos. Essa flexibilidade foi crucial para lidar com a complexidade da ferramenta, permitindo refinamento da solução a cada iteração. Além disso, a arquitetura do sistema foi guiada pelos princípios do *Domain-Driven Design* (DDD) (EVANS, 2004), garantindo que a lógica de negócio (o domínio da regressão simbólica) permanecesse isolada das complexidades de infraestrutura, como banco de dados e brokers de mensagens, o que permite a substituição de implementações específicas por outras desde que a interface definida pela aplicação seja respeitada.

3.2 AMBIENTE DE DESENVOLVIMENTO E FERRAMENTAS

O projeto foi desenvolvido utilizando o Windows Subsystem for Linux (WSL), uma ferramenta que permite a criação de máquinas virtuais Linux dentro de um computador Windows, e Docker, uma ferramenta que permite a containerização de aplicações.

O Docker foi utilizado para facilitar o gerenciamento do ambiente de desenvolvimento back-end, permitindo a criação de dois serviços essenciais para a aplicação: Postgres (Banco de Dados Relacional) e RabbitMQ (Broker de Mensagens). O uso dessa ferramenta permite a replicabilidade do ambiente de desenvolvimento em qualquer máquina, facilitando o processo de criação e execução do código.

3.2.1 DESENVOLVIMENTO FRONT-END

A construção das telas da plataforma e suas funcionalidades foi feita através do *framework* React (desenvolvido e mantido pela Meta) que utiliza a linguagem imperativa *JavaScript*. Além disso, as bibliotecas *Tanstack React Query* e *Zustand* foram utilizadas, respectivamente, para gerenciamento de estado assíncrono, ou seja, informações vindas diretamente do back-end da aplicação, e para gerenciamento do estado interno da aplicação front-end.

Demais ferramentas utilizadas e suas versões específicas podem ser encontradas abaixo:

Código 3.1 – Bibliotecas utilizadas no front-end e suas versões

```
1 "dependencies": {
2   "@hookform/resolvers": "^5.2.2",
3   "@monaco-editor/react": "^4.8.0-rc.3",
4   "@react-router/node": "^7.9.2",
5   "@react-router/serve": "^7.9.2",
6   "@tanstack/react-query": "^5.90.21",
7   "@tanstack/react-query-devtools": "^5.91.3",
8   "@tanstack/react-table": "^8.21.3",
9   "clsx": "^2.1.1",
10  "framer-motion": "^12.35.2",
11  "isbot": "^5.1.31",
12  "lucide-react": "^0.577.0",
13  "papaparse": "^5.5.3",
14  "react": "^19.1.1",
15  "react-dom": "^19.1.1",
16  "react-dropzone": "^14.3.8",
17  "react-hook-form": "^7.69.0",
18  "react-intersection-observer": "^10.0.3",
19  "react-katex": "^3.1.0",
20  "react-markdown": "^10.1.0",
21  "react-router": "^7.9.2",
22  "rehype-highlight": "^7.0.2",
23  "remark-gfm": "^4.0.1",
24  "tailwind-merge": "^3.4.0",
25  "uuid": "^13.0.0",
26  "zod": "^4.3.6",
27  "zustand": "^5.0.11"
28 },
29 "devDependencies": {
30   "@react-router/dev": "^7.9.2",
31   "@tailwindcss/typography": "^0.5.19",
32   "@tailwindcss/vite": "^4.1.13",
33   "@types/node": "^22",
34   "@types/papaparse": "^5.5.2",
35   "@types/react": "^19.1.13",
36   "@types/react-dom": "^19.1.9",
```

```

37     "@types/react-katex": "^3.0.4",
38     "tailwindcss": "^4.1.13",
39     "typescript": "^5.9.2",
40     "vite": "^7.1.7",
41     "vite-tsconfig-paths": "^5.1.4"
42 }

```

Para permitir o treinamento de modelos de regressão de forma assíncrona, evitando o bloqueio do servidor HTTP, o sistema utiliza *workers* independentes em Python que se comunicam com a API principal através do *RabbitMQ*, integrado via biblioteca *aio_pika*. Por fim, as operações de regressão simbólica foram feitas utilizando as bibliotecas *eggp* para o treinamento, e *rEGGression* para manipulação dos modelos. O *PostgreSQL* foi o banco de dados escolhido devido à natureza fortemente estruturada e relacional das entidades do sistema.

3.2.2 ESTRUTURA DO PROJETO

A organização do código-fonte segue uma estrutura modular para facilitar a manutenção e o desacoplamento entre os componentes. Abaixo é apresentada a árvore de diretórios principal:

```

spectrum/
├── docker/ ..... Configurações de containers (Postgres, RabbitMQ)
├── packages/
│   ├── backend/ ..... API FastAPI e lógica de aplicação
│   ├── core/ ..... Domínio puro, entidades e interfaces (ports)
│   ├── db_core/ ..... Implementação de repositórios e SQLAlchemy
│   ├── frontend/ ..... Interface React e gerenciamento de estado
│   ├── inference_query_language/ ..... Parser e executor da IQL
│   └── workers/ ..... Processamento especializado assíncrono
├── paper/ ..... Código-fonte LaTeX da monografia
└── scripts/ ..... Utilitários de desenvolvimento e automação

```

Cada pacote possui uma responsabilidade bem definida: o *core* isola a lógica de negócio da aplicação, definindo as entidades principais do projeto e as interfaces necessárias para manipulação dessas entidades; o *db_core* lida exclusivamente com a persistência, definindo as tabelas do banco de dados, bem como as interfaces responsáveis pela manipulação dos dados por meio de operações intermediadas pelo banco de dados; o *backend* orquestra as requisições HTTP e a comunicação com os *workers* via *RabbitMQ*; o *frontend* provê a camada de interação com o usuário final; e o *inference_query_language* isola a lógica da linguagem de domínio IQL, utilizada para interagir com os modelos de regressão

via sintaxe similar ao SQL.

Demais ferramentas utilizadas e suas versões específicas podem ser encontradas abaixo:

Código 3.2 – Ferramentas utilizadas no back-end e suas versões

```
// Servidor
dependencies = [
    "aio-pika>=9.5.8",
    "aiofiles>=25.1.0",
    "alembic>=1.18.3",
    "asyncpg>=0.31.0",
    "core",
    "db-core",
    "dotenv>=0.9.9",
    "eggpy>=1.0.16",
    "fastapi[standard]>=0.128.0",
    "inference-query-language",
    "pandas>=3.0.0",
    "pika>=1.3.2",
    "pydantic[email]>=2.12.5",
    "pydantic-settings>=2.12.0",
    "python-json-logger>=4.0.0",
    "regression>=1.0.10",
    "sqlalchemy>=2.0.46",
    "uvicorn[standard]",
    "argon2-cffi>=25.1.0",
    "jose>=1.0.0",
    "python-jose[cryptography]>=3.5.0",
]

[tool.uv]
dev-dependencies = [
    "pytest",
    "pytest-asyncio",
    "httpx",
    "ruff",
    "mypy",
    "pre-commit",
]

// Pacote Core
dependencies = [
```

```
"inference-query-language",
"pydantic>=2.12.5",
"pydantic-settings>=2.12.0",
"regression>=1.0.10",
]

// Pacote DB Core
dependencies = [
    "core",
]

// Pacote IQL
dependencies = [
    "pandas>=3.0.0",
    "pydantic>=2.12.5",
]

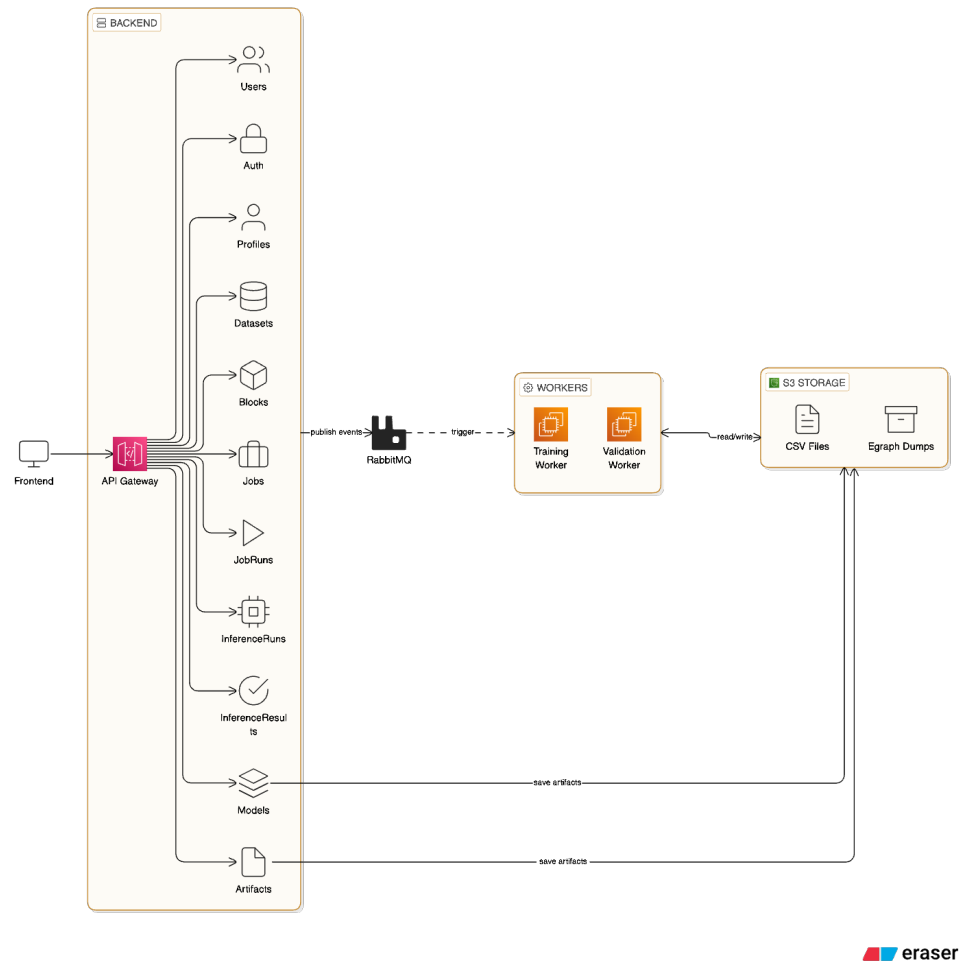
// Pacote Workers
dependencies = [
    "eggp>=1.0.16",
    "core",
    "db-core",
    "aio-pika>=9.5.8",
]
```

3.3 ARQUITETURA

A arquitetura da plataforma foi projetada sob o modelo cliente-servidor (SOMMERVILLE, 2011) com o objetivo de oferecer desacoplamento entre a interface gráfica (front-end) utilizada pelo usuário e a lógica de processamento, ou regras de negócio, e persistência de dados (back-end), permitindo a evolução independente de ambos os sistemas.

Nesta seção, é apresentado o fluxo geral de integração dos componentes do sistema, detalhando os padrões de projeto adotados, baseando-se nos princípios do *Domain Driven Design* (EVANS, 2004), uma forma de desenvolvimento de *software* que preza pelo desenvolvimento dentro de domínios específicos, ou seja, dentro de micro-universos que representam as regras de negócio da aplicação. No capítulo seguinte será apresentado os detalhes de implementação da plataforma, bem como suas principais funcionalidades e também pontos de melhoria e exploração futuros.

Figura 7 – Diagrama de alto nível da arquitetura do projeto.



3.3.1 VISÃO GERAL E FLUXO DE DADOS

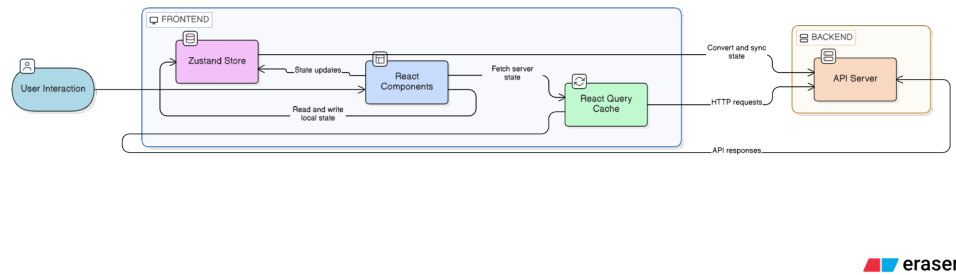
O diagrama 7 apresenta a arquitetura de alto nível do projeto, dividida em quatro principais componentes:

1. Front-end
2. Back-end
3. Workers
4. Armazenamento

3.3.1.1 FRONT-END

O front-end é responsável por mostrar ao usuário os dados vindos da API (*Application Programming Interface*) e permitir a manipulação desses dados. Por meio da interface, o usuário pode:

Figura 8 – Diagrama de alto nível da arquitetura do front-end.



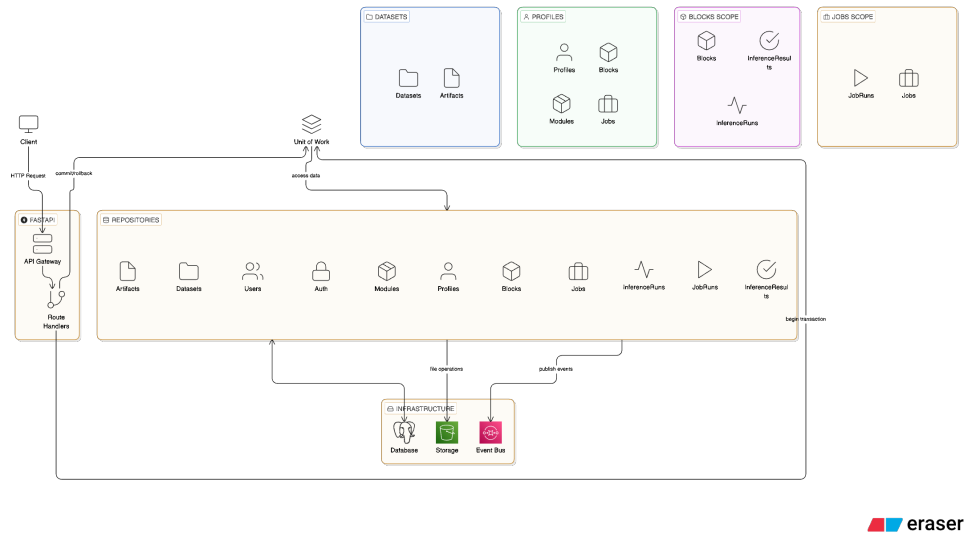
1. Fazer upload de conjunto de dados
2. Criar *Profiles*, ou seja, espaços nos quais os conjuntos de dados podem ser importados para treinamento e manipulação de modelos
3. Criar modelos de regressão simbólica para os conjuntos de dados de uma *Profile*
4. Gerenciar (atualizar e/ou excluir) conjuntos de dados e *Profiles*
5. Criar uma conta na plataforma
6. Fazer login na conta criada
7. Escrever scripts IQL para manipulação dos modelos criados
8. Observar, por meio de elementos visuais como gráficos, propriedades dos modelos criados

Toda comunicação com o back-end é feita de forma assíncrona, por meio de requisições HTTP ou *Server Side Events (SSE)*, os quais permitem o back-end se comunicar diretamente com o front-end enviando dados via *Streaming*. O front-end é implementado utilizando-se duas bibliotecas para gerenciar o estado da aplicação. O estado interno, ou seja, informações contidas apenas no front-end é gerenciado pela biblioteca *Zustand*, enquanto o estado assíncrono, ou seja, informações vindas do back-end, é gerenciado pela biblioteca *Tanstack React Query*, a qual também oferece funcionalidades de cache e controle de estado de requisições HTTP, como estados de carregamento e erro. A imagem 8 apresenta um diagrama de alto nível da arquitetura do front-end, mostrando o fluxo de dados e a divisão de responsabilidades entre os componentes.

3.3.1.2 BACK-END

O back-end é responsável pelas regras de negócio da aplicação. Ele foi desenvolvido de forma modular, ou seja, cada módulo apresenta suas próprias regras de negócio e

Figura 9 – Diagrama de alto nível da arquitetura do back-end.



interfaces de comunicação próprias. É de responsabilidade do back-end também a comunicação com os demais serviços da plataforma, como os serviços de armazenamento (local ou S3, por exemplo), brokers de mensagens e *workers*.

O diagrama 9 mostra o fluxo de uma requisição HTTP na aplicação, bem como a divisão de escopos para as entidades presentes. O principal padrão estrutural do back-end é o *Unit of Work*, o qual possibilita retornar o estado da aplicação ao estado inicial caso um fluxo de mudanças encontre um erro. Esse padrão se utiliza de transações de banco de dados para garantir integridade do sistema (MARTIN, 2008). As transações são controladas implicitamente pelo fluxo da aplicação Python, sendo automaticamente realizada a operação de *commit* assim que o escopo da função responsável por determinada funcionalidade se encerra, ou sendo realizado o *rollback* caso uma exceção seja levantada em código. Toda manipulação que necessita do banco de dados é realizada por meio de um repositório, o qual é uma classe que define operações base para cada entidade do sistema.

3.3.1.3 WORKERS

Os *workers* são responsáveis por treinar os modelos de regressão utilizando a biblioteca *eggpy* (FRANÇA; KRONBERGER, 2025a), e por validar os modelos criados, ou seja, calcular a fronteira de pareto do modelo e calcular os valores gerados por cada expressão de pareto. Assim como o *back-end*, as manipulações ao banco de dados são feitas via repositórios. O padrão *Unit of Work* também é utilizado, mas, diferentemente do back-end, o controle das transações é feito de forma manual.

Cada *worker* consome eventos específicos da fila de mensagens de forma assíncrona. Como cada *worker* atualiza as informações pertinentes para seu domínio, não há comunicação no sentido dos *workers* para o *back-end*. No entanto, *workers* podem se comunicar

com a fila de mensagens, emitindo novos eventos para si mesmo ou outros *workers*.

3.3.1.4 ARMAZENAMENTO

O componente de armazenamento é responsável por armazenar os dados que não podem ser salvos no banco de dados, ou seja, artefatos necessários para o funcionamento da aplicação que não podem ser armazenados de forma tabular e relacional. Os principais elementos guardados nesse componente são:

1. Arquivos CSV para os conjuntos de dados que cada usuário fez upload
2. Arquivos para os e-graphs gerados pelo treinamento dos modelos
3. Arquivos CSV gerados pelo treinamento dos modelos

O componente de armazenamento apresenta dois modos de operação: modo local e modo remoto. No modo local, os arquivos são armazenados no sistema de arquivos do servidor que hospeda a aplicação. No modo remoto, os arquivos são armazenados em um S3, serviço da AWS para armazenar arquivos.

A interface de armazenamento é definida de forma abstrata, por meio de protocolos, no pacote *core*:

Código 3.3 – Definição da interface de armazenamento

```
from typing import Protocol, BinaryIO
from .upload_result import UploadResult

class FileStorage(Protocol):
    async def upload(
        self,
        *,
        path: str,
        file: BinaryIO,
    ) -> UploadResult: ...

    async def download(
        self,
        *,
        path: str,
        destination: str,
    ) -> None: ...
```

Uma implementação para essa interface deve definir os métodos *upload* e *download*. Cada componente da aplicação que acessa o componente de armazenamento define sua própria implementação, permitindo que detalhes de implementação sejam escondidos do restante da aplicação que necessita utilizar esse componente. Esse padrão é chamado de *Ports and Adapters* e permite a troca de implementações de serviços específicos dentro da aplicação desde que os métodos necessários sejam oferecidos por cada implementação. Abaixo, segue exemplo de implementação do serviço de armazenamento para o *back-end*:

Código 3.4 – Implementação da interface de armazenamento no back-end

```
import hashlib

from typing import BinaryIO
from pathlib import Path

from app.core.config import settings

from core.ports.storage.file_storage import FileStorage
from core.ports.storage.path import SpectrumPath
from core.ports.storage.upload_result import UploadResult


class LocalStorage(FileStorage):
    def __init__(self) -> None:
        self._base_path = Path(
            settings.FILE_STORAGE_SPECTRUM_DATA_PATH
        )

    async def upload(
        self,
        *,
        path: str,
        file: BinaryIO
    ) -> UploadResult:
        full_path = self._base_path / path
        full_path.parent.mkdir(parents=True, exist_ok=True)

        hasher = hashlib.sha256()
        size = 0

        file.seek(0)
```

```

with open(full_path, 'wb') as f:
    while chunk := file.read(8192):
        f.write(chunk)
        hasher.update(chunk)
        size += len(chunk)

spectrum_path = SpectrumPath(
    full_path=full_path, path=Path(path))
checksum = hasher.hexdigest()

return UploadResult(
    path=spectrum_path,
    size=size,
    checksum=checksum
)

async def download(
    self,
    *,
    path: str,
    destination: str
) -> None:
    source_path = self._base_path / path
    destination_path = Path(destination)

    if not source_path.exists():
        raise FileNotFoundError(
            f"File not found at path: {source_path}")

    destination_path.parent.mkdir(
        parents=True, exist_ok=True)
    destination_path.write_bytes(source_path.read_bytes())

```

3.3.2 ABSTRAÇÕES E CONCEITOS BASE

Os pacotes **core** e **db_core** definem as principais abstrações utilizadas pelo sistema como forma de simplificar implementações e definir interfaces de comunicação claras e precisas que permitam a substituição de subsistemas caso necessário, isolando subsiste-

mas do mundo exterior, de acordo com o padrão de código *Ports and Adapters* (MARTIN, 2008).

3.3.2.1 ABSTRAÇÕES DE BANCO DE DADOS

O pacote `db_core` utiliza as bibliotecas *SQLAlchemy* e *Alembic* para gerenciar as entidades do banco de dados. Para facilitar a implementação das entidades, foram definidas algumas classes utilitárias para padrões comuns:

1. **Entidades Mutáveis:** Entidades que podem ser alteradas de forma direta pelo usuário da aplicação. Suportam operações de escrita, leitura e deleção (CRUD). Definido pelo código 3.5. Toda entidade mutável apresenta as propriedades *id*, um identificador único gerado pelo padrão UUIDv4; *created_at*, um marcador temporal de quando a entidade foi criada no sistema; e *updated_at*, um marcador temporal de quando a entidade foi alterada no sistema;
2. **Entidades Imutáveis:** Entidades que não podem ser alteradas diretamente pelo usuário, podendo ser alteradas apenas pela própria plataforma Spectrum. Suportam apenas operações de criação e leitura. Definido pelo código 3.6;
3. **Entidades Versionadas:** Entidades que são imutáveis mas apresentam versões diferentes. Versões antigas são mantidas para consulta. Suportam apenas operações de criação e leitura. Definida pelo código 3.7

Código 3.5 – Definição de uma entidade mutável em nível de banco de dados

```
from uuid import UUID
from datetime import datetime

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import func, DateTime, text
from sqlalchemy.dialects.postgresql import UUID as PG_UUID

from .root import RootBase

class MutableBase(RootBase):
    __abstract__ = True

    id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        primary_key=True,
```

```

        server_default=text("gen_random_uuid()"),
    )

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
        server_onupdate=func.now(),
    )

```

Código 3.6 – Definição de uma entidade imutável em nível de banco de dados

```

from typing import Any
from uuid import UUID
from datetime import datetime

from sqlalchemy.orm import Mapped, mapped_column, declared_attr
from sqlalchemy import func, DateTime, UniqueConstraint, Index,
    Boolean, text
from sqlalchemy.dialects.postgresql import UUID as PG_UUID

from .root import RootBase

class ImmutableBase(RootBase):
    __abstract__ = True

    id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        primary_key=True,
        server_default=text("gen_random_uuid()"),
    )

    timestamp: Mapped[datetime] = mapped_column(

```



```

        DateTime( timezone=True) ,
        nullable=False ,
        server_default=func.now() ,
    )

@declared_attr.directive
def __table_args__(cls) -> Any:
    return (
        UniqueConstraint("id", name=f"uq_{cls.__tablename__}"
                           "_id"),
        Index(f"idx_{cls.__tablename__}_timestamp", "
              timestamp"),
    )

```

Código 3.7 – Definição de uma entidade versionada em nível de banco de dados

```

class ImmutableVersionedBase(ImmutableBase):
    __abstract__ = True

    id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        primary_key=True,
        server_default=text("gen_random_uuid()"),
    )

    version: Mapped[int] = mapped_column(
        nullable=False,
        primary_key=True,
        default=1,
    )

    is_latest: Mapped[bool] = mapped_column(
        Boolean,
        default=True,
        nullable=False,
    )

    timestamp: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
    )

```

```

server_default=func.now(),
)

@declared_attr.directive
def __table_args__(cls) -> Any:
    return (
        UniqueConstraint(
            "id", "version", name=f"uq_{cls.__tablename__}
            _id_version"),
        Index(f"idx_{cls.__tablename__}_id_version", "id", "
            version"),
        Index(f"idx_{cls.__tablename__}_is_latest", "
            is_latest"),
        Index(f"idx_{cls.__tablename__}_latest", "id",
            unique=True, postgresql_where=text("is_latest
            = true"))
    )

```

3.3.3 ABSTRAÇÕES DE DOMÍNIO

O pacote **core** define as entidades de domínio do sistema, bem como interfaces comuns aos demais pacotes. As representações de domínio seguem uma estrutura similar àquelas definidas para o banco de dados. No entanto, as entidades de domínio não possuem detalhes de implementação relacionados à persistência, como anotações de tipo específicos para o banco de dados, índices ou restrições de chave estrangeira. Além disso, segundo as práticas do Código Limpo ([MARTIN, 2008](#)), entidades de domínio não precisam equivaler-se diretamente às entidades de banco de dados, podendo apresentar uma estrutura diferente, desde que as regras de negócio sejam respeitadas.

Assim como as entidades de banco de dados, as entidades de domínio também são divididas em mutáveis (código 3.8), imutáveis (código 3.9) e versionadas (código 3.10). Essas entidades são implementadas utilizando-se a biblioteca *Pydantic*, a qual permite a definição de modelos de dados com validação de dados acoplada, bem como facilita a conversão entre diferentes representações de dados, como dicionários, JSON e objetos Python.

Código 3.8 – Definição de uma entidade mutável em nível de domínio

```

class BaseMutableDomainModel(BaseDomainModel):
    id: UUID = Field(default_factory=uuid4)
    created_at: datetime = Field(default_factory=lambda:
        datetime.now(timezone.utc))

```

```
updated_at: datetime = Field(default_factory=lambda:
    datetime.now(timezone.utc))
```

Código 3.9 – Definição de uma entidade imutável em nível de domínio

```
class BaseImmutableDomainModel(RootDomainModel):
    id: UUID = Field(default_factory=uuid4)
    timestamp: datetime = Field(default_factory=lambda: datetime
        .now(timezone.utc))
```

Código 3.10 – Definição de uma entidade versionada em nível de domínio

```
class BaseImmutableVersionedDomainModel(RootDomainModel):
    id: UUID = Field(default_factory=uuid4)
    version: int = Field(default=1, description="Version number
        for optimistic concurrency control")
    timestamp: datetime = Field(default_factory=lambda: datetime
        .now(timezone.utc))
    is_latest: bool = Field(default=True)
```

Para além das definições bases para entidades de domínio, o pacote **core** também define os protocolos para os repositórios. Segundo as práticas do Código Limpo (MARTIN, 2008), repositórios são classes responsáveis por encapsular a lógica necessária para acessar fontes de dados, como bancos de dados, serviços externos ou mesmo arquivos locais, isolando o restante da aplicação dos detalhes de implementação relacionados à persistência de dados. As implementações específicas dos protocolos estão presentes no pacote **db_core**.

A definição dos protocolos de repositórios segue uma estrutura similar àquela das entidades de domínio, apresentando uma divisão entre repositórios para entidades mutáveis (código 3.11), imutáveis (código 3.12) e versionadas (código 3.13). Além disso, também é definido um protocolo para repositórios que permitem operações em lote (código 3.14), ou seja, operações que manipulam múltiplas entidades de forma simultânea.

Para garantir a segurança de tipos em tempo de compilação, os protocolos são implementados utilizando-se da funcionalidade de *Generics* do Python, permitindo a definição de tipos genéricos para as entidades manipuladas pelos repositórios, bem como para os filtros e cláusulas de busca utilizadas nas operações de leitura. Dessa forma, é possível garantir que as implementações específicas dos repositórios respeitem as interfaces definidas pelos protocolos, evitando erros de tipo em tempo de execução.

Código 3.11 – Definição de um protocolo de repositório para entidades mutáveis

```
from typing import Protocol
```

```

from core.features.base import (
    BaseMutableDomainModel,
)

from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter

from .immutable import IImmutableRepository

class IMutableRepository[
    T_Mutable: BaseMutableDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](IImmutableRepository[T_Mutable, T_Where, T_Filter], Protocol):
    async def update(self, entity: T_Mutable) -> None: ...
    async def delete(self, entity: T_Mutable) -> None: ...

```

Código 3.12 – Definição de um protocolo de repositório para entidades imutáveis

```

from typing import Optional, Protocol, Sequence

from core.features.base import (
    RootDomainModel,
)

from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter
from core.utils.pagination.response import PaginatedResponse
from core.utils.pagination.base import Pagination

class IImmutableRepository[
    T_Immutable: RootDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](Protocol):
    async def add(self, entity: T_Immutable) -> None: ...
    async def get_unique(self, where: T_Where) -> Optional[
        T_Immutable]: ...

```

```

async def list(self, filter: Optional[T_Filter] = None,
               pagination: Optional[Pagination] = None) ->
               PaginatedResponse[T_Immutable]: ...

async def list_all(
    self, filter: Optional[T_Filter] = None) -> Sequence[
    T_Immutable]: ...

```

Código 3.13 – Definição de um protocolo de repositório para entidades versionadas

```

from typing import Optional, Protocol
from uuid import UUID

from core.features.base import BaseImmutableVersionedDomainModel
from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter

from .immutable import ImmutableRepository

class IVersionedRepository[
    T_Versioned: BaseImmutableVersionedDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](ImmutableRepository[T_Versioned, T_Where, T_Filter], Protocol
):
    async def get_latest_version(
        self, parent_id: UUID) -> Optional[T_Versioned]: ...

    async def get_version_by_number(
        self, parent_id: UUID, version: int) -> Optional[
        T_Versioned]: ...

    async def unset_latest_and_add(
        self, parent_id: UUID, new_entity: T_Versioned) -> None:
        ...

```

Código 3.14 – Definição de um protocolo de repositório para operações em lote

```

from typing import Protocol, Iterable

```

```

from core.features.base import (
    BaseMutableDomainModel,
)

from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter

from .mutable import IMutableRepository

class IMutableBulkRepository[
    T_Mutable: BaseMutableDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](IMutableRepository[T_Mutable, T_Where, T_Filter], Protocol):
    async def add_many(self, entities: Iterable[T_Mutable]) ->
        None: ...
    async def update_many(self, entities: Iterable[T_Mutable])
        -> None: ...
    async def delete_many(self, entities: Iterable[T_Mutable])
        -> None: ...

```

Por fim, o pacote **core** também define protocolos para o padrão de código *Unit of Work*, definindo quais componentes podem ser controlados dentro de uma determinada unidade de trabalho. O padrão de código *Unit of Work* é um padrão de design que tem como objetivo manter um registro de todas as operações realizadas durante uma transação, garantindo que todas as operações sejam concluídas com sucesso ou revertidas em caso de falha, mantendo a integridade dos dados (MARTIN, 2008). Em conjunto com a definição do protocolo de unidades de trabalho, a plataforma também trabalha com o conceito de recursos transacionais, ou seja, recursos que precisam ser controlados dentro de uma unidade de trabalho para garantir a integridade do sistema, mas que não são associados a transações de banco de dados. Os principais recursos transacionais utilizados são conexões com brokers de mensagens e conexões com serviços de armazenamento de dados.

As definições para o protocolo de unidade de trabalho podem ser encontrados no código 3.15, enquanto as definições para o protocolo de recurso transacional podem ser encontradas no código 3.16.

Código 3.15 – Definição de um protocolo para o padrão de código Unit of Work

```

class UnitOfWork(ABC):
    users: IUsersRepository

```

```

auth: IAuthRepository
datasets: IDatasetsRepository
artifacts: IArtifactsRepository
profiles: IProfilesRepository
blocks: IBlocksRepository
jobs: IJobsRepository
models: IModelsRepository
runs: IRunsRepository
inference_runs: IIInferenceRunRepository
inference_results: IIInferenceResultRepository

file_storage: FileStorage
events_publisher: EventPublisher

def __init__(self) -> None:
    self._resources: list[TransactionalResource] = []
    self._on_commit_hooks: list[Callable[[], Awaitable[None]]] = []

def register(self, resource: TransactionalResource) -> None:
    self._resources.append(resource)

async def commit_resources(self) -> None:
    for resource in self._resources:
        await resource.commit()

    self._resources.clear()

async def rollback_resources(self) -> None:
    for resource in self._resources:
        await resource.rollback()

    self._resources.clear()

@abstractmethod
async def __aenter__(self) -> UnitOfWork: ...

@abstractmethod
async def __aexit__(self, exc_type: Optional[Type[

```

```

        BaseException]],
        exc: Optional[BaseException],
        tb: Optional[TracebackType],): ...

@abstractmethod
async def commit(self) -> None: ...

def on_commit(self, hook: Callable[[], Awaitable[None]]) ->
    None:
    self._on_commit_hooks.append(hook)

@abstractmethod
async def rollback(self) -> None: ...

```

Código 3.16 – Definição de um protocolo para recursos transacionais

```

class TransactionalResource(Protocol):
    async def commit(self) -> None:
        ...

    async def rollback(self) -> None:
        ...

```


4 PLATAFORMA WEB PARA EXPLORAÇÃO DE ALGORITMOS DE REGRESSÃO SIMBÓLICA

4.1 PANORAMA GERAL

A plataforma proposta nesse projeto tem como objetivo principal a manipulação de modelos de regressão simbólica em *pipelines* de extração e processamento de dados. A plataforma pode ser separada em três principais componentes:

1. Backend: Servidor responsável por processar as requisições Web, gerenciar o banco de dados e os modelos de regressão simbólica;
2. Worker: Programa especializado para treinamento dos modelos de regressão; e
3. Frontend: Aplicação web que permite o usuário interagir com os modelos criados e as demais entidades do projeto.

Para entender melhor as funcionalidades esperadas da plataforma, primeiro foram elencados os requisitos funcionais e não funcionais do projeto. Segundo o DDD, os requisitos funcionais são aqueles que dizem respeito a ações que podem ser realizadas dentro do sistema desenvolvido. Em outras palavras, os requisitos funcionais dizem o que uma aplicação é capaz de fazer. Por outro lado, os requisitos não funcionais dizem como um sistema opera, delimitando aspectos de segurança, restrições e desempenho da aplicação.

4.2 HISTÓRIAS DE USUÁRIO

Como os requisitos funcionais dizem respeito às ações que podem ser realizadas dentro de uma aplicação, eles foram descritos por meio de Histórias de Usuário (*User Stories*), técnica amplamente utilizada em metodologias ágeis (COHN, 2004). Essas histórias descrevem um fluxo de operação do sistema sob a perspectiva do ator final, permitindo entender as funcionalidades e o valor de cada entrega.

Além disso, como um dos objetivos do projeto é a criação de uma plataforma de manipulação de algoritmos de regressão simbólica que tenha paridade com as ferramentas comerciais disponíveis (como TuringBot e HeuristicLab), incorporou-se funcionalidades consolidadas no mercado juntamente com as inovações propostas pelo Spectrum (como

a linguagem IQL). Dessa forma, o projeto aqui descrito possui os seguintes requisitos funcionais, estratificados pelas seguintes histórias de usuário principais:

- **US1 - Upload de Dados:** Como usuários, quero fazer *upload* de arquivos CSV para que eu possa utilizar meus próprios dados experimentais no treinamento de modelos.
- **US2 - Datasets Aleatórios:** Como usuários, quero gerar conjuntos de dados aleatórios para testar o comportamento dos algoritmos em diferentes cenários matemáticos.
- **US3 - Treinamento de Modelos:** Como usuário, quero configurar os meta-parâmetros do algoritmo Eggp para que eu possa otimizar a busca por expressões que respeitem restrições específicas de domínio.
- **US4 - Consulta IQL:** Como analista de dados, quero consultar expressões encontradas através da linguagem IQL para identificar padrões matemáticos de interesse.
- **US5 - Visualização de Métricas:** Como usuários, quero observar métricas de performance (MSE, R^2) e complexidade estrutural para facilitar a seleção do modelo mais adequado.
- **US6 - Predição e Validação:** Como usuário, quero utilizar os modelos treinados para realizar previsões em novos dados, validando a generalização da solução encontrada.

4.3 INTERFACE WEB E BLOCOS DE VISUALIZAÇÃO

A plataforma Spectrum utiliza uma interface baseada em "blocos" para visualização e manipulação de dados, permitindo que a interface se adapte às necessidades de cada perfil de usuário. Cada perfil pode ser personalizado com blocos modulares, como por exemplo:

- **Bloco de Editor IQL:** Provê uma interface de codificação com realce de sintaxe para execução de consultas declarativas sobre os modelos de regressão;
- **Bloco de Markdown:** Permite a documentação textual diretamente na plataforma, integrando anotações do pesquisador aos experimentos realizados.

A interface em blocos se assemelha a outras plataformas amplamente utilizadas em ambientes de ciência de dados, como o Google Colab. Dessa forma, a plataforma proposta se aproxima de outras ferramentas já utilizadas atualmente, facilitando a utilização

dela. Além disso, o objetivo dessa interface modular é integrar as etapas de exploração, escrita e utilização dos modelos de regressão. Ferramentas disponíveis hoje em dia, como HeuristicLab e TuringBot, não permitem a documentação do modelo de forma integrada (HEURISTICLAB TEAM, 2013; TURINGBOT SOFTWARE, 2020), implicando na necessidade de um pesquisador utilizar diversas ferramentas simultaneamente e acarretando em trocas de contexto constantes.

4.4 REQUISITOS NÃO-FUNCIONAIS

Os requisitos não-funcionais definem as restrições e qualidades do sistema Spectrum. Para esta plataforma, os seguintes requisitos foram estabelecidos:

1. **Segurança:** A autenticação deve ser realizada via JSON Web Tokens (JWT) armazenados em cookies *HttpOnly* para mitigar ataques de Cross-Site Scripting (XSS);
2. **Desempenho:** O treinamento de modelos, por ser uma tarefa intensiva em termos de CPU, deve ser executado de forma assíncrona em processos separados (*workers*);
3. **Integridade:** Todas as mutações no banco de dados devem seguir o padrão *Unit of Work*, garantindo a atomicidade das transações;
4. **Portabilidade:** O sistema deve ser capaz de ser executado em diferentes ambientes por meio de containerização com Docker.

4.5 BACK-END

O back-end do projeto foi desenvolvido seguindo o padrão monólito modularizado, em oposição a uma arquitetura de micro-serviços pura, visando reduzir a complexidade de rede sem abrir mão da separação de responsabilidades. O servidor é responsável por gerenciar as principais entidades do sistema:

- **User:** Representa os usuários da plataforma;
- **Dataset:** Metadados e referências aos arquivos CSV de dados;
- **Job:** Representa uma configuração para o algoritmo eggp, contendo os meta parâmetros do algoritmo;
- **Run:** O resultado de um treinamento, contendo metadados como tempo de execução;
- **Profile:** Personalização da interface e blocos de visualização para o usuário;

- **Model:** Representa um modelo gerado por um **Job**, com referências para o arquivo egraph gerado;
- **Block:** Representa um bloco no editor;
- **Inference Run:** Representa uma requisição em linguagem IQL para determinada **Profile**;
- **Inference Result:** Representa os resultados de uma consulta IQL.

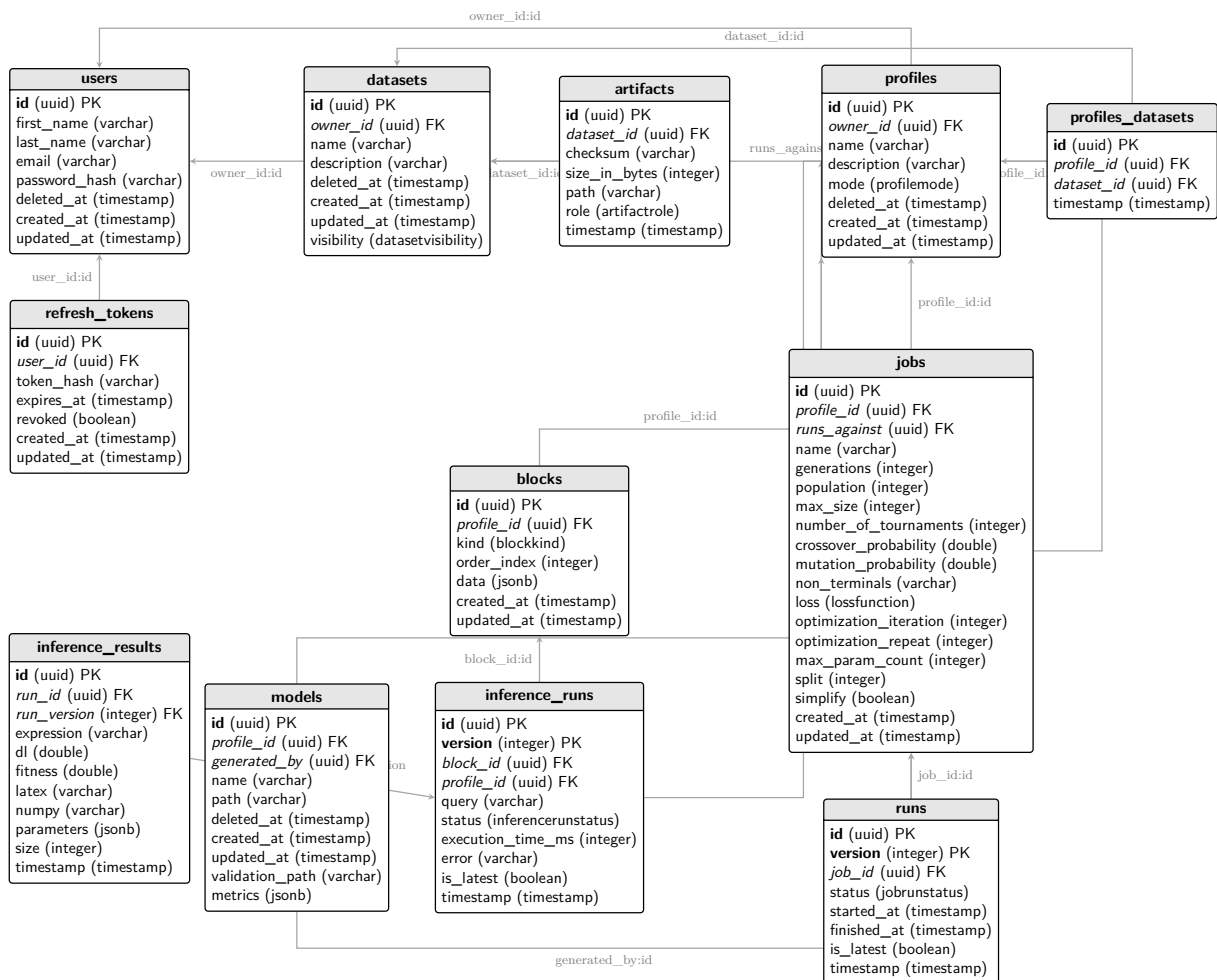


Figura 10 – Diagrama de entidades do sistema

A figura 10 apresenta a estrutura de entidades do sistema, bem como suas relações. As sub-seções seguintes apresentam detalhes de cada módulo, suas regras de negócios, as rotas HTTP disponibilizadas por eles manipulação do seu domínio, bem como os valores esperados de entrada e saída para cada rota.

4.5.1 USUÁRIOS

Como forma de permitir a criação de modelos privados, a plataforma Spectrum oferece uma funcionalidade de criação de contas de usuário. Todas as demais entidades do

sistema estão relacionadas, direta ou indiretamente, com um usuário. Usuários representam pessoas que utilizam a plataforma. Um usuário é definido a nível de banco de dados pelo código 4.1, e a nível de domínio pelo código 4.2.

Código 4.1 – Código SQLAlchemy para criação da tabela de usuários

```
class UserORM( MutableBase ):
    __tablename__ = " users "

    first_name: Mapped[ str ] = mapped_column( String , nullable=
        False )
    last_name: Mapped[ str ] = mapped_column( String , nullable=
        False )

    email: Mapped[ str ] = mapped_column( String , unique=True ,
        nullable=False )
    password_hash: Mapped[ str ] = mapped_column( String , nullable=
        False )

    deleted_at: Mapped[ datetime | None ] = mapped_column(
        DateTime( timezone=True ) , nullable=True )

    datasets: Mapped[ list [ "DatasetORM" ] ] = relationship(
        "DatasetORM" ,
        back_populates="owner" ,
        cascade="all , delete-orphan" ,
        lazy="selectin" ,
    )

    profiles: Mapped[ list [ "ProfileORM" ] ] = relationship(
        "ProfileORM" ,
        back_populates="owner" ,
        cascade="all , delete-orphan" ,
        lazy="selectin" ,
    )
```

Código 4.2 – Classe de domínio que define um usuário

```
class User( BaseMutableDomainModel ):
    first_name: str = Field( ... , description="The user's first
        name" )
```

```

last_name: str = Field(..., description="The user's last name")

email: EmailStr = Field(..., description="The user's email address")

password_hash: SecretStr = Field(...,
                                   description="The hashed password of the user")

deleted_at: Optional[datetime] = Field(
    None, description="The timestamp when the user was deleted, if applicable")

@classmethod
def new(
    cls,
    *,
    first_name: str,
    last_name: str,
    email: EmailStr,
    password_hash: SecretStr,
):
    return cls(
        first_name=first_name,
        last_name=last_name,
        email=email,
        password_hash=password_hash,
        deleted_at=None,
    )

```

Usuários podem manipular outras duas principais entidades do sistema: **Datasets** e **Profiles**. Datasets são abstrações sobre os arquivos CSV de dados que os usuários fazem upload para a plataforma, enquanto Profiles são espaços de trabalho contendo blocos de visualização e modelos de regressão simbólica. Dessa forma, a plataforma garante que cada usuário tenha acesso apenas aos seus próprios dados e modelos, promovendo a privacidade e segurança das informações.

4.5.2 DATASETS

Datasets são uma abstração sobre os arquivos CSV de dados que os usuários fazem upload para a plataforma. Além de referências para os arquivos, por meio das entidades de Artefatos, os Datasets também armazenam metadados como nome, descrição, data de criação e usuário proprietário. Um Dataset representa um conjunto de arquivos de dados com funções diferentes, como por exemplo um conjunto de dados de treinamento e outro de validação. A plataforma permite que os usuários façam upload de múltiplos arquivos para um mesmo Dataset, facilitando a organização dos dados experimentais. O código 4.3 apresenta a classe de domínio que define um Dataset e suas propriedades e métodos.

Código 4.3 – Classe de domínio que define um Dataset

```
class Dataset(BaseMutableDomainModel):
    name: str = Field()
    description: str = Field()
    owner_id: UUID = Field()
    artifacts: list[Artifact] = Field()
    deleted_at: datetime | None = Field()

    visibility: DatasetVisibility = Field(
        DatasetVisibility.PRIVATE,
    )

    @classmethod
    def new(
        cls,
        name: str,
        description: str,
        owner_id: UUID,
        visibility: DatasetVisibility = DatasetVisibility.
            PRIVATE
    ) -> "Dataset":
        return cls(
            name=name,
            description=description,
            owner_id=owner_id,
            artifacts=[],
            deleted_at=None,
            visibility=visibility
        )
```

Datasets possuem uma propriedade que controla sua visibilidade na plataforma. Normalmente, datasets são privados, permitindo que apenas o usuário que criou o dataset tenha acesso a ele. Caso a visibilidade seja definida como pública, qualquer usuário da plataforma pode importar esse dataset em sua profile. No entanto, as funcionalidades de editar e deletar o dataset continua sendo restringida apenas para o usuário proprietário, garantindo a integridade dos dados mesmo em casos de compartilhamento. Essa funcionalidade de compartilhamento de datasets é fundamental para promover a colaboração entre os usuários da plataforma, permitindo que eles compartilhem conjuntos de dados experimentais e promovam a comparação e análise de resultados entre diferentes experimentos de regressão simbólica.

Outra regra de negócio importante presente no modelo de domínio do Dataset é a prática do *soft delete*, presente na propriedade *deleted_at* que pode ser tanto vazia quanto um marcador de tempo. Esse mecanismo diz que uma entidade não pode ser deletada do banco de dados, mas deve ser marcada como deletada, seja por meio de uma *flag* ou pela marcação do horário em que a exclusão aconteceu. Essa prática permite que os usuários possam recuperar datasets deletados, além de manter o histórico de entidades criadas por um usuário, mesmo que elas tenham sido deletadas posteriormente.

4.5.3 PROFILES

Profiles são espaços de trabalho personalizados para cada usuário, contendo blocos de visualização e modelos de regressão simbólica. Cada Profile pode ser configurado com diferentes blocos, como o Editor IQL para consultas e um bloco de Markdown para documentação. Essa personalização permite que os usuários adaptem a interface às suas necessidades específicas, integrando a exploração, escrita e utilização dos modelos de regressão em um único ambiente. Além disso, os Profiles estão diretamente relacionados aos modelos de regressão simbólica, permitindo que os usuários organizem seus experimentos de forma estruturada e eficiente. O código 4.4 apresenta a classe de domínio que define um Profile e suas propriedades e métodos.

Código 4.4 – Classe de domínio que define um Profile

```
class Profile(BaseMutableDomainModel):
    name: str = Field()
    description: str = Field()
    owner_id: UUID = Field()
    mode: ProfileMode = Field(default=ProfileMode.DRAFT)
    deleted_at: datetime | None = Field(default=None)
    datasets: list[Dataset] = Field(default=[])
    jobs: list[Job] = Field(default=[])
    models: list[Model] = Field(default=[])
```



```

blocks: list[Block] = Field(default=[])

@classmethod
def new(
    cls,
    name: str,
    description: str,
    owner_id: UUID,
    mode: ProfileMode = ProfileMode.DRAFT
) -> "Profile":
    return cls(
        name=name,
        description=description,
        owner_id=owner_id,
        mode=mode,
        deleted_at=None,
        datasets=[],
        jobs=[],
        models=[],
        blocks=[]
    )

```

Dentro da plataforma proposta, Profiles são o ponto de agregação de diversas outras entidades, como Jobs, Runs e Models. Dessa forma, cada Profile pode conter múltiplos Jobs (configurações de treinamento), cada Job pode gerar múltiplos Runs (resultados de treinamento) e cada Run pode estar associado a um Model (modelo de regressão simbólica gerado). Essa estrutura hierárquica facilita a organização dos experimentos e a navegação entre diferentes etapas do processo de modelagem.

Para criar um Job dentro de uma determinada Profile, o usuário deve associar ao Profile um Dataset. Dessa forma, um mesmo Profile pode conter diversos datasets, cada um associado a um Job. Essa flexibilidade permite que os usuários experimentem diferentes conjuntos de dados, ou até o mesmo conjunto com meta-parâmetros completamente diferentes, dentro do mesmo espaço de trabalho, promovendo a comparação e análise de resultados de forma integrada.

As profiles contém uma propriedade que controla o modo de apresentação delas, o qual pode ser definido como *Draft* ou *Published*. Enquanto uma profile estiver em modo *Draft*, apenas o usuário dono da profile pode visualizá-la. Já quando a profile é publicada, ela pode ser visualizada por outros usuários, promovendo a colaboração e compartilhamento de experimentos entre os usuários da plataforma. No entanto, mesmo quando uma

profile é publicada, apenas o usuário dono da profile tem permissão para editar ou deletar a profile, garantindo a integridade dos dados e experimentos mesmo em casos de compartilhamento.

4.5.3.1 JOBS

Jobs representam configurações para o algoritmo *EGGP*, contendo os parâmetros do algoritmo. Cada Job possui uma associação com um Dataset específico, permitindo que os usuários configurem o treinamento de modelos de regressão simbólica com diferentes conjuntos de dados. Os Jobs são o ponto de partida para a execução do treinamento, e cada execução gera um Run, que contém os resultados do treinamento, como tempo de execução e métricas de performance. A plataforma estabelece um requisito para os datasets associados a um Job, exigindo que eles contenham pelo menos um artefato do tipo *Data* e um artefato do tipo *Validation*, garantindo que os *workers* que atuam sobre o conjunto de dados consigam criar um modelo de regressão simbólica e validá-lo de forma adequada. A imagem 4.5 apresenta a classe de domínio que define um Job e suas propriedades e métodos.

Código 4.5 – Classe de domínio que define um Job

```
class Job(BaseMutableDomainModel):
    name: str = Field()
    profile_id: UUID = Field()
    runs_against: UUID = Field()
    generations: int = Field()
    population: int = Field()
    max_size: int = Field()
    number_of_tournaments: int = Field()
    crossover_probability: float = Field()
    mutation_probability: float = Field()
    non_terminals: list[AvailableFunction] = Field()
    loss: LossFunction = Field()
    optimization_iterations: int = Field()
    optimization_repeats: int = Field()
    max_param_count: int = Field()
    split: int = Field()
    simplify: bool = Field()
    runs: list[Run] = Field([])

    @classmethod
    def new(
```

```

        cls ,
        name: str ,
        profile_id: UUID,
        runs_against: UUID,
        generations: int ,
        population: int ,
        max_size: int ,
        number_of_tournaments: int ,
        crossover_probability: float ,
        mutation_probability: float ,
        non_terminals: list [ AvailableFunction ] ,
        loss: LossFunction ,
        optimization_iterations: int ,
        optimization_repeats: int ,
        max_param_count: int ,
        split: int ,
        simplify: bool ,
    ) -> "Job":
        return cls(
            name=name,
            profile_id=profile_id ,
            runs_against=runs_against ,
            generations=generations ,
            population=population ,
            max_size=max_size ,
            number_of_tournaments=number_of_tournaments ,
            crossover_probability=crossover_probability ,
            mutation_probability=mutation_probability ,
            non_terminals=non_terminals ,
            loss=loss ,
            optimization_iterations=optimization_iterations ,
            optimization_repeats=optimization_repeats ,
            max_param_count=max_param_count ,
            split=split ,
            simplify=simplify ,
            runs=[],
        )

```

Atualmente, a plataforma contém uma lista estática de funções disponíveis para serem utilizadas como funções de perda e como não terminais. Dessa forma, os usuários

podem escolher entre as funções disponíveis para configurar seus Jobs, mas não podem criar suas próprias funções personalizadas. As listas de funções de perda e de não terminais podem ser observadas nos códigos 4.6 e 4.7, respectivamente.

Código 4.6 – Lista de funções de perda disponíveis

```
class LossFunction(StrEnum):  
    MSE = "MSE"  
    GAUSSIAN = "Gaussian"  
    BERNOULLI = "Bernoulli"  
    POISSON = "Poisson"
```

Código 4.7 – Lista de não terminais disponíveis

```
class AvailableFunction(StrEnum):  
    ADD = "add"  
    SUB = "sub"  
    MUL = "mul"  
    DIV = "div"  
  
    POWER = "power"  
    POWERABS = "powerabs"  
    SQUARE = "square"  
    CUBE = "cube"  
  
    Sqrt = "sqrt"  
    SqrtABS = "sqrtabs"  
    CBRT = "cbrt"  
  
    SIN = "sin"  
    COS = "cos"  
    TAN = "tan"  
    ASIN = "asin"  
    ACOS = "acos"  
    ATAN = "atan"  
  
    SINH = "sinh"  
    COSH = "cosh"  
    TANH = "tanh"  
    ASINH = "asinh"  
    ACOSH = "acosh"  
    ATANH = "atanh"
```

```

ABS = "abs"
LOG = "log"
LOGABS = "logabs"
EXP = "exp"
RECIP = "recip"
AQ = "aq"

@staticmethod
def from_list(*functions: AvailableFunction) -> str:
    return ", ".join(functions)

@staticmethod
def to_list(functions_str: str) -> list[AvailableFunction]:
    return [AvailableFunction(func.strip()) for func in
            functions_str.split(",")]

@staticmethod
def default() -> list[AvailableFunction]:
    return [
        AvailableFunction.ADD,
        AvailableFunction.SUB,
        AvailableFunction.MUL,
        AvailableFunction.DIV,
    ]

```

4.5.3.2 RUNS

Runs representam os resultados de um treinamento, contendo metadados como tempo de execução e métricas de performance. Cada Run está associado a um Job específico, permitindo que os usuários acompanhem o progresso e os resultados de diferentes configurações de treinamento. Os Runs são essenciais para a análise dos modelos gerados, pois fornecem informações sobre a eficácia do treinamento e a qualidade dos modelos de regressão simbólica encontrados. Além disso, os Runs estão diretamente relacionados aos Models, que representam os modelos de regressão simbólica gerados a partir de cada execução, permitindo que os usuários naveguem entre os resultados e os modelos de forma integrada. O código 4.8 apresenta a classe de domínio que define um Run e suas propriedades e métodos.

Código 4.8 – Classe de domínio que define um Run

```

class Run(BaseImmutableVersionedDomainModel):
    job_id: UUID = Field()
    status: JobRunStatus = Field(JobRunStatus.WAITING)
    started_at: datetime | None = Field(default_factory=lambda:
        None)
    finished_at: datetime | None = Field(default_factory=lambda:
        None)

    @classmethod
    def new(
        cls,
        id: UUID | None,
        job_id: UUID,
        version: int,
    ) -> "Run":
        if id is None:
            id = uuid4()

        return cls(
            id=id,
            job_id=job_id,
            version=version,
            status=JobRunStatus.WAITING,
            is_latest=True,
        )

```

Dentro da plataforma, Runs são armazenadas de forma versionada, ou seja, executar um mesmo Job múltiplas vezes gera múltiplas Runs com o mesmo identificador, mas com versões diferentes, permitindo que os usuários acompanhem a evolução dos resultados ao longo do tempo. Essa abordagem facilita a comparação entre diferentes execuções do mesmo Job, promovendo a análise de como mudanças nos meta-parâmetros ou nos conjuntos de dados impactam os resultados do treinamento. Além disso, essa forma de armazenamento permite que os usuários mantenham um histórico completo de suas experimentações, facilitando a rastreabilidade e a reprodutibilidade dos resultados obtidos.

4.5.3.3 MODELS

Models são uma abstração sobre modelos de regressão simbólica criados pelo algoritmo Eggp. Cada Model está associado a um Run específico, representando o modelo gerado a partir de uma execução de treinamento (Job). Os Models contêm referências

para os arquivos e-graph gerados durante o processo de treinamento, permitindo que os usuários acessem e manipulem os modelos de regressão simbólica de forma eficiente. Além disso, os Models estão diretamente relacionados aos Profiles, permitindo que os usuários organizem seus experimentos e modelos dentro de seus espaços de trabalho personalizados. O código 4.9 apresenta a classe de domínio que define um Model e suas propriedades e métodos.

Código 4.9 – Classe de domínio que define um Model

```
class Model(BaseMutableDomainModel):
    name: str = Field()
    profile_id: UUID = Field()
    generated_by: UUID = Field()
    path: str = Field()
    validation_path: str | None = Field()
    metrics: dict[str, Any] | None = Field()
```

Na plataforma Spectrum, Models são tratados apenas como entidades de negócio, não representando o artefato em si, mas sim uma abstração sobre ele. Dessa forma, um Model contém metadados como nome, descrição, data de criação e referências para os arquivos e-graph gerados, mas não armazena diretamente o conteúdo do modelo de regressão simbólica. Essa abordagem permite que os usuários manipulem os Models de forma eficiente, sem a necessidade de lidar diretamente com os arquivos e-graph, promovendo uma experiência de usuário mais fluida e integrada. Além disso, essa abstração facilita a integração de diferentes backends de regressão simbólica no futuro, como o PySR, sem a necessidade de mudanças significativas na estrutura de dados dos Models, promovendo a flexibilidade e escalabilidade da plataforma.

A entidade Model, além de armazenar as informações referentes ao modelo, armazena em si a sua fronteira de pareto, permitindo que os usuários acessem e analisem as expressões presentes na fronteira de pareto de forma eficiente por meio da interface web da plataforma. Essa funcionalidade é fundamental para a análise comparativa entre diferentes expressões, permitindo que os usuários identifiquem quais expressões apresentam melhor desempenho em dados não vistos durante o treinamento, promovendo a seleção de expressões mais adequadas e generalizáveis para suas aplicações específicas. As predições de cada expressão da fronteira de pareto, no entanto, é armazenada em um arquivo CSV referenciado na propriedade *validation_path*.

4.5.3.4 BLOCKS, INFERENCERUNS, INFERENCERESULTS

Além de armazenarem jobs, resultados e modelos, os Profiles também armazenam os blocos de visualização e as consultas em linguagem IQL. Dessa forma, cada Profile

pode conter múltiplos Blocks. Cada Block representa um bloco de visualização na interface, como o Editor IQL ou um bloco de Markdown para documentação. Os Blocks estão diretamente relacionados às InferenceRuns, que representam as consultas em linguagem IQL realizadas pelos usuários. Cada InferenceRun está associada a um Block específico, permitindo que os usuários organizem suas consultas e resultados de forma estruturada dentro de seus Profiles. Além disso, cada InferenceRun gera múltiplos InferenceResults, que representam os resultados das consultas em linguagem IQL, permitindo que os usuários acessem e analisem os resultados de suas consultas de forma eficiente e integrada à interface da plataforma. O código 4.10 apresenta a classe de domínio que define um Block e suas propriedades e métodos.

Código 4.10 – Classe de domínio que define um Block

```
class Block(BaseMutableDomainModel):
    profile_id: UUID = Field()
    kind: BlockKind = Field()
    order_index: int = Field()
    data: BaseBlock = Field()

    @classmethod
    def new(cls, profile_id: UUID, kind: BlockKind, order_index:
        int, data: BaseBlock):
        return cls(
            profile_id=profile_id,
            kind=kind,
            order_index=order_index,
            data=data
        )
```

Um bloco de visualização pode ser tanto do tipo *Inference*, representando um bloco de consulta em linguagem IQL (como mostrado no código 4.11 e na imagem 22), quanto do tipo *Markdown*, representando um bloco de documentação em linguagem Markdown (como mostrado no código 4.12).

Código 4.11 – Classe de domínio que define um bloco de consulta em linguagem IQL

```
class BaseBlock[T](BaseModel):
    data: T
    kind: Any

class InferenceBlock(BaseBlock[str]):
    kind: Literal[BlockKind.INFERENCE] = BlockKind.INFERENCE
```


Código 4.12 – Classe de domínio que define um bloco de consulta em linguagem IQL

```
class MarkdownBlock(BaseBlock[str]):
    kind: Literal[BlockKind.MARKDOWN] = BlockKind.MARKDOWN
```

Blocos do tipo *Inference* podem estar associados a diversas *InferenceRuns*, entidades que representam os pedidos de execução de consultadas definidas naquele bloco. O código 4.13 apresenta a classe de domínio que define uma *InferenceRun* e suas propriedades e métodos.

Código 4.13 – Classe de domínio que define uma *InferenceRun*

```
class InferenceRun(BaseImmutableVersionedDomainModel):
    block_id: UUID = Field()
    profile_id: UUID = Field()
    query: str = Field()
    status: InferenceRunStatus = Field(InferenceRunStatus.
        PENDING)
    execution_time_ms: Optional[int] = Field(None)
    error: Optional[str] = Field(None)

    @classmethod
    def new(cls, block_id: UUID, profile_id: UUID, query: str,
        version: int = 1):
        return cls(
            block_id=block_id,
            profile_id=profile_id,
            query=query,
            version=version,
            is_latest=True,
            status=InferenceRunStatus.PENDING,
            execution_time_ms=None,
            error=None
        )
```

As propriedades de uma *InferenceRun* são atualizadas conforme a execução da consulta IQL é realizada. A seção 4.6.3 e o capítulo sobre a linguagem de consulta apresentam em mais detalhes o processo de execução. Quando a execução de uma consulta é finalizada, os resultados gerados são armazenados em uma entidade do tipo *InferenceResult*, que contém as expressões encontradas e as métricas de performance associadas a cada expressão. O código 4.14 apresenta a classe de domínio que define uma *InferenceResult* e suas propriedades e métodos.

Código 4.14 – Classe de domínio que define uma InferenceResult

```
class InferenceResult(BaseImmutableDomainModel):
    run_id: UUID = Field()
    run_version: int = Field()
    expression: str = Field()
    dl: Optional[float] = Field(None)
    fitness: Optional[float] = Field(None)
    latex: Optional[str] = Field(None)
    numpy: Optional[str] = Field(None)
    parameters: Optional[dict[str, float]] = Field(None)
    size: Optional[int] = Field(None)

    @classmethod
    def new(
        cls,
        run_id: UUID,
        run_version: int,
        expression: str,
        dl: Optional[float] = None,
        fitness: Optional[float] = None,
        latex: Optional[str] = None,
        numpy: Optional[str] = None,
        parameters: Optional[dict[str, float]] = None,
        size: Optional[int] = None
    ):
        return cls(
            run_id=run_id,
            run_version=run_version,
            expression=expression,
            dl=dl,
            fitness=fitness,
            latex=latex,
            numpy=numpy,
            parameters=parameters,
            size=size
        )
```

4.6 WORKERS

Na plataforma proposta neste projeto, *workers* são programas especializados para executar tarefas intensivas em termos de CPU, como treinamento e validação de modelos de regressão simbólica, bem como a execução de consultas em linguagem IQL. Esses programas especializados operam de forma assíncrona, permitindo que o servidor principal do sistema seja responsável apenas por gerenciar as requisições Web, o banco de dados e a lógica de negócios, enquanto os *workers* lidam com as tarefas computacionalmente intensivas. Essa arquitetura de execução remota é fundamental para garantir o desempenho e a escalabilidade da plataforma, permitindo que usuários com diferentes níveis de poder computacional possam utilizar a ferramenta de forma eficiente.

A comunicação entre o servidor principal e os *workers* é realizada por meio de uma fila de mensagens, utilizando RabbitMQ como sistema de mensageria. Essa abordagem permite que as tarefas sejam enfileiradas e processadas de forma assíncrona, garantindo a responsividade do sistema mesmo durante a execução de tarefas intensivas. Além disso, essa arquitetura facilita a escalabilidade da plataforma, permitindo que múltiplos *workers* sejam adicionados conforme a demanda, garantindo que os usuários possam realizar seus treinamentos e consultas de forma eficiente, independentemente do poder computacional disponível localmente.

Cada *worker* é especializado em uma tarefa específica e está associado a uma chave de roteamento na fila de mensagens. A medida que as requisições são recebidas pelo servidor principal, elas são enfileiradas com a chave de roteamento correspondente à tarefa a ser executada, e os *workers* monitoram a fila para processar as tarefas conforme elas chegam.

4.6.1 TREINAMENTO

A principal limitação das ferramentas atualmente disponíveis é a necessidade de execução local do processo de treinamento, o que pode ser um empecilho para usuários que não possuem o poder computacional necessário. Como forma de superar essa limitação, a plataforma Spectrum implementa um sistema de execução remota para o treinamento de modelos de regressão simbólica.

O processo de treinamento é iniciado quando um usuário faz uma requisição HTTP para o servidor principal com o objetivo de criar um novo Run a partir de um Job específico. O servidor principal, então, emite uma mensagem na fila de mensagens com a chave de roteamento associada ao treinamento, contendo as informações necessárias para a execução do treinamento. Todos os *workers* possuem acesso ao banco de dados, portanto, as mensagens enviadas contém apenas os identificadores das entidades necessárias para a execução do treinamento.

Dessa forma, o *worker* responsável por processar a tarefa de treinamento monitora a fila de mensagens para identificar quando uma nova tarefa de treinamento é enfileirada. Ao receber a mensagem, o *worker* consulta o banco de dados para obter as informações necessárias sobre o Job e o Dataset associados. Após obter essas informações, o arquivo do tipo *Data* é baixado pelo *worker* para ser utilizado como conjunto de dados de treinamento. Com o arquivo armazenado localmente, o *worker* cria uma nova instância do algoritmo Eggp, configurando-o com os meta-parâmetros especificados no Job. O treinamento é então enviado para ser executado em um thread separado com o objetivo de não bloquear a comunicação com a fila de mensagens, permitindo que o *worker* continue processando outras tarefas enquanto o treinamento está em andamento. Durante a execução do treinamento, o *worker* monitora o progresso e, ao final do processo, armazena os resultados no banco de dados, incluindo as métricas de performance e referências para os arquivos e-graph gerados. Ao longo do treinamento, o *worker* também é responsável por atualizar o status do Run no banco de dados, permitindo que os usuários acompanhem o progresso do treinamento em tempo real por meio da interface web da plataforma.

4.6.2 VALIDAÇÃO

Além do treinamento, a plataforma Spectrum executa um processo denominado validação, que consiste em avaliar a fronteira de pareto do modelo gerado durante o treinamento utilizando um conjunto de dados diferente do conjunto de treinamento. Esse processo é fundamental para garantir a generalização dos modelos de regressão simbólica encontrados, permitindo que os usuários tenham confiança na aplicabilidade dos modelos em dados não vistos durante o treinamento.

O processo de validação é iniciado automaticamente após a conclusão do treinamento, no qual o *worker* de treinamento emite uma mensagem na fila de mensagens com a chave de roteamento associada à validação. O *worker* responsável por processar a tarefa de validação monitora a fila de mensagens para identificar quando uma nova tarefa de validação é enfileirada. Ao receber a mensagem, o *worker* consulta o banco de dados para obter as informações necessárias sobre o Run e o Dataset associados. Após obter essas informações, o arquivo do tipo *Validation* é baixado pelo *worker* para ser utilizado como conjunto de dados de validação. Com o arquivo armazenado localmente, o *worker* cria uma instância rEGGression para consultar as informações do e-graph gerado. Primeiramente, a fronteira de pareto é calculada. Então, o *worker* executa a validação dos modelos presentes na fronteira de pareto do modelo gerado durante o treinamento, calculando as métricas de performance utilizando o conjunto de dados de validação. Os resultados da validação são então armazenados no banco de dados, permitindo que os usuários acessem e analisem as métricas de performance dos modelos em seus Profiles. Além disso, o processo de validação é fundamental para a análise comparativa entre diferentes expressões, per-

mitindo que os usuários identifiquem quais expressões apresentam melhor desempenho em dados não vistos durante o treinamento, promovendo a seleção de expressões mais adequadas e generalizáveis para suas aplicações específicas.

4.6.3 CONSULTAS IQL

Como forma de interagir com expressões presentes no e-graph de forma declarativa, a plataforma Spectrum implementa um sistema de execução remota para consultas em linguagem IQL. Esse sistema segue uma arquitetura similar àquela utilizada para o treinamento e validação, utilizando a fila de mensagens para processar as consultas de forma assíncrona. Quando um usuário realiza uma consulta em linguagem IQL por meio do Editor IQL na interface web, o servidor principal emite uma mensagem na fila de mensagens com a chave de roteamento associada à execução de consultas IQL, contendo as informações necessárias para a execução da consulta. O *worker* responsável por processar as consultas IQL monitora a fila de mensagens para identificar quando uma nova tarefa de consulta é enfileirada. Ao receber a mensagem, o *worker* consulta o banco de dados para obter as informações necessárias sobre a InferenceRun e o Profile associados. Após obter essas informações, o *worker* cria uma nova instância de rEGGression para consultar as informações do e-graph gerado durante o treinamento. A consulta em linguagem IQL é então executada de forma assíncrona, permitindo que o *worker* continue processando outras tarefas enquanto a consulta está em andamento.

O motor de execução de consultas IQL é responsável por converter o texto da consulta nas funções e parâmetros necessários para se utilizar a instância de rEGGression (FRANÇA; KRONBERGER, 2025b). Ao receber a consulta, o motor de execução converte o texto em uma sequência de tokens, utilizando um processo de tokenização. Em seguida, a sequência de tokens é transformada em uma árvore de sintaxe abstrata (AST) por meio de um processo de parsing. A AST é então interpretada para extrair as informações necessárias para a execução da consulta, como os campos a serem retornados, as condições de filtragem e os padrões de busca. Com essas informações, o motor de execução constrói as chamadas apropriadas para a instância de rEGGression, executando a consulta sobre o e-graph e obtendo os resultados correspondentes. Os resultados da consulta são então armazenados no banco de dados como InferenceResults, permitindo que os usuários acessem e analisem os resultados de suas consultas IQL por meio da interface web da plataforma.

Associado à execução remota das consultas IQL, a plataforma também implementa um sistema de streaming de resultados, permitindo que os usuários visualizem em tempo real as etapas de execução de suas consultas. À medida que o motor de execução processa a consulta, os resultados parciais, ao serem atualizados pelo *worker*, são recuperados pelo servidor central e enviados para a interface web por meio de Server Side Events (SSE).

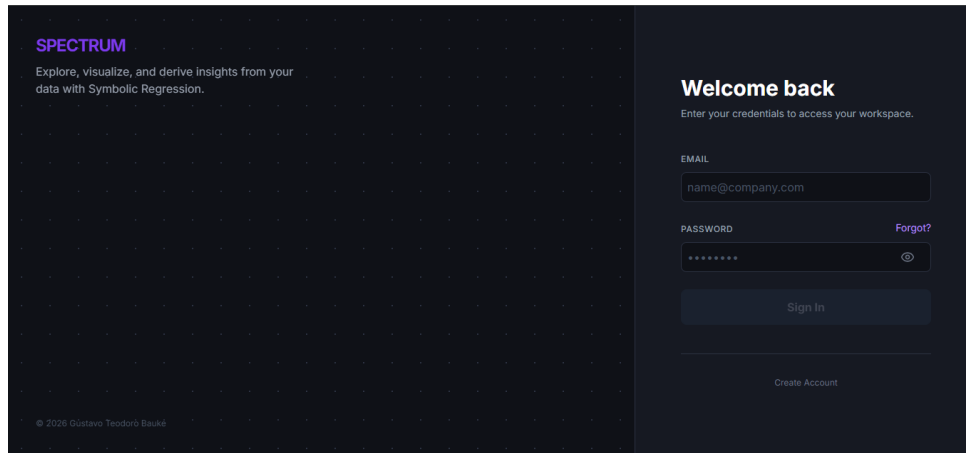


Figura 11 – Tela de login da plataforma Spectrum

Essa abordagem de streaming de resultados proporciona uma experiência de usuário mais fluida e interativa, permitindo que os usuários acompanhem o progresso de suas consultas em tempo real, identificando rapidamente os resultados à medida que são gerados, promovendo uma análise mais dinâmica e eficiente dos modelos de regressão simbólica presentes no e-graph.

4.7 FRONT-END

O front-end da plataforma Spectrum é uma aplicação web desenvolvida utilizando React, que permite aos usuários interagir com os modelos de regressão simbólica e as demais entidades do sistema por meio de uma interface intuitiva e personalizada. A interface é baseada em blocos de visualização, permitindo que os usuários configurem seus espaços de trabalho de acordo com suas necessidades específicas, integrando a exploração, escrita e utilização dos modelos de regressão em um único ambiente. O front-end se comunica com o back-end por meio de requisições HTTP para manipular as entidades do sistema, como Users, Datasets, Profiles, Jobs, Runs e Models, bem como para enviar consultas em linguagem IQL e receber os resultados correspondentes. Além disso, o front-end também é responsável por exibir as métricas de performance dos modelos de regressão simbólica, permitindo que os usuários analisem e comparem os resultados de forma eficiente por meio da interface web da plataforma.

4.7.1 TELA DE LOGIN

A imagem 11 apresenta a tela de login da plataforma Spectrum, onde os usuários podem inserir suas credenciais (email e senha) para acessar seus espaços de trabalho personalizados. A autenticação é realizada por meio de JSON Web Tokens (JWT) armazenados em cookies *HttpOnly*, garantindo a segurança das informações dos usuários e mitigando ataques de Cross-Site Scripting (XSS). A tela de login é o ponto de entrada

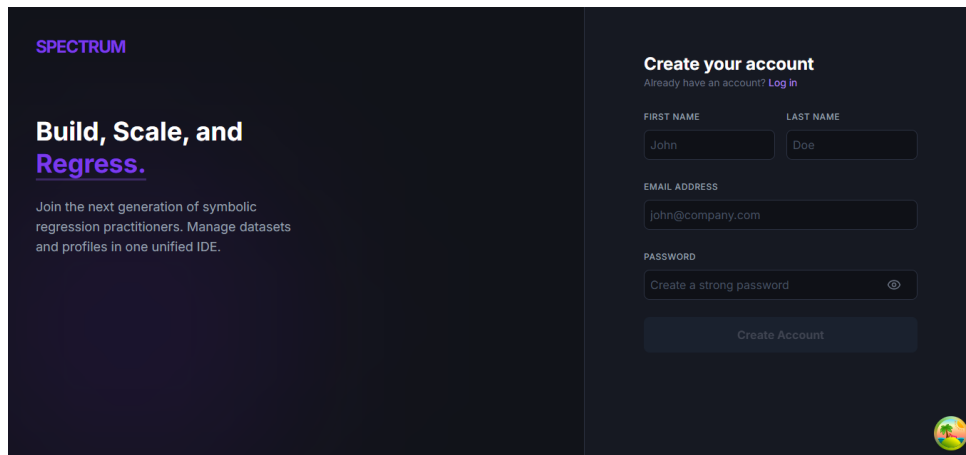


Figura 12 – Tela de cadastro da plataforma Spectrum

para a plataforma, permitindo que os usuários acessem suas contas e comecem a explorar as funcionalidades oferecidas pela ferramenta.

4.7.2 TELA DE CADASTRO

A imagem 12 apresenta a tela de cadastro da plataforma Spectrum, onde os usuários podem criar uma nova conta e acessar seus espaços de trabalho. Para criar uma conta, os usuários devem fornecer informações como nome, sobrenome, email e senha. O processo de cadastro é fundamental para garantir a privacidade e segurança das informações dos usuários, permitindo que cada usuário tenha acesso apenas aos seus próprios dados e modelos de regressão simbólica. A tela de cadastro é o ponto de entrada para novos usuários na plataforma, permitindo que eles criem suas contas e comecem a explorar as funcionalidades oferecidas pela ferramenta.

4.7.3 EDITOR

O fluxo principal de interação dos usuários com a plataforma ocorre por meio do Editor, onde os usuários podem navegar entre seus Profiles e Datasets, configurar Jobs para treinamento, observar resultados de treinamento e validação, e realizar consultas em linguagem IQL. O Editor é o espaço de trabalho central da plataforma, integrando todas as funcionalidades oferecidas pela ferramenta em uma interface intuitiva e personalizada. Por meio do Editor, os usuários podem organizar seus experimentos, analisar os resultados dos modelos de regressão simbólica e interagir com os modelos presentes no e-graph de forma eficiente, promovendo uma experiência de usuário fluida e integrada para a exploração de algoritmos de regressão simbólica. A imagem 13 apresenta a tela inicial do Editor.

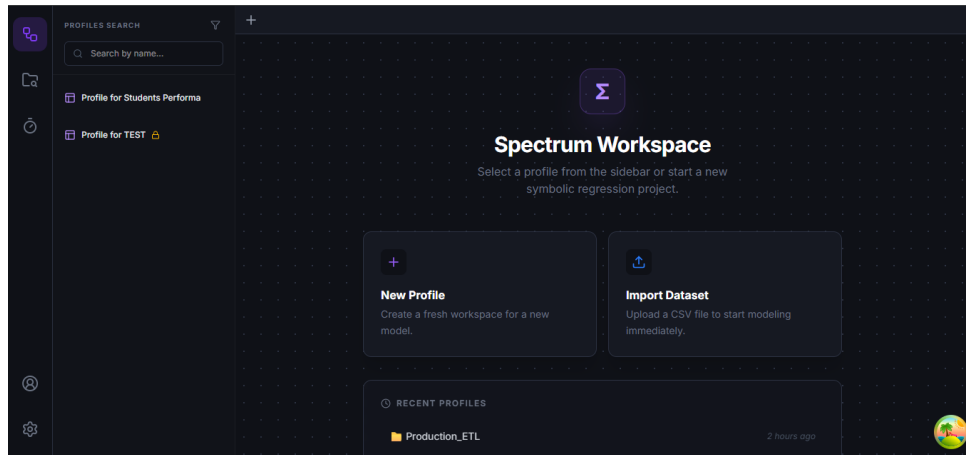


Figura 13 – Tela inicial do Editor da plataforma Spectrum

4.7.3.1 NAVEGAÇÃO

A tela do editor contém três principais elementos de navegação e uma área central de atividades. Os dois primeiros elementos de navegação são a barra de atividades, a qual apresenta, por meio de ícones, as principais funcionalidades da plataforma, e a barra lateral, que apresenta visões diferentes de acordo com a funcionalidade selecionada na barra de atividades. Por exemplo, ao selecionar a funcionalidade de Profiles, a barra lateral apresenta uma visão geral dos Profiles do usuário, permitindo que ele navegue entre seus espaços de trabalho personalizados. Já ao selecionar a funcionalidade de Datasets, a barra lateral apresenta uma visão geral dos Datasets do usuário, permitindo que ele gerencie seus conjuntos de dados experimentais. A área central do Editor é onde os usuários podem configurar Jobs para treinamento, observar resultados de treinamento e validação, e realizar consultas em linguagem IQL, integrando todas as funcionalidades oferecidas pela plataforma em um único espaço de trabalho intuitivo e eficiente para a exploração de algoritmos de regressão simbólica. Além disso, a tela do Editor também apresenta uma área de navegação na parte superior na qual Profiles abertas são apresentadas como abas para facilitar a navegação entre diferentes espaços de trabalho. A imagem 14 apresenta os elementos referentes à barra de atividades e à barra lateral.

A barra de atividades apresenta duas principais visões: Profiles e Datasets. A visão de Profiles permite que os usuários naveguem entre seus espaços de trabalho, enquanto a visão de Datasets permite que os usuários gerenciem seus conjuntos de dados. Ambas as visões apresentam implementação semelhante, utilizando a biblioteca *React Query* para realizar o gerenciamento do estado e cache das informações vindas do back-end. A imagem 14 apresenta a barra lateral com a visão de Profiles, enquanto a imagem 15 apresenta a barra lateral com a visão de Datasets.

Outra funcionalidade importante presente na visão de Datasets é o botão de importar um dataset em específico para uma Profile. Ao clicar nesse botão, caso o usuário

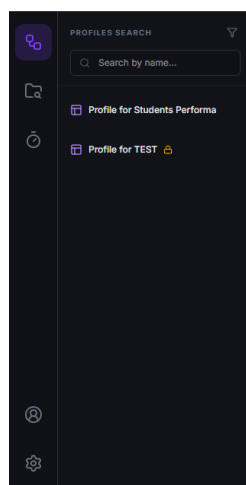


Figura 14 – Elementos de navegação da barra de atividades e da barra lateral do Editor da plataforma Spectrum

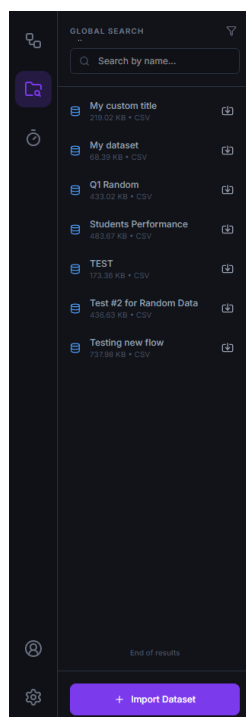


Figura 15 – Barra lateral do Editor da plataforma Spectrum com a visão de Datasets

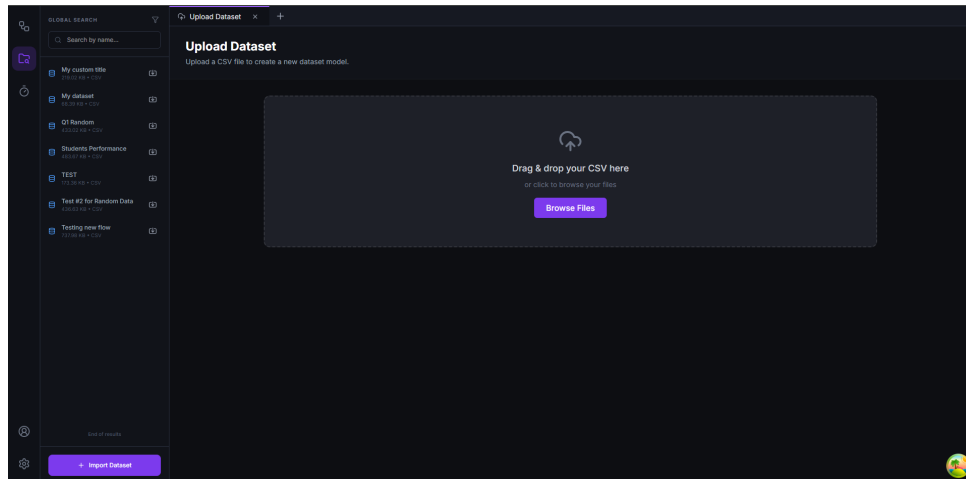


Figura 16 – Tela de upload de datasets da plataforma Spectrum

tenha uma profile aberta, o dataset é associado à profile, permitindo que o usuário configure um Job para treinamento utilizando os dados do dataset importado. Caso nenhuma profile esteja ativa no editor, uma nova aba é aberta para uma nova profile com o dataset associado.

Além disso, a visão de datasets apresenta um botão que permite a criação de um novo dataset, o qual, ao ser clicado, redireciona o usuário para a página de upload de datasets, a qual é implementada também como uma aba do editor. A imagem 16 apresenta a tela de upload de datasets, onde os usuários podem fazer upload de arquivos CSV para serem utilizados em seus experimentos de regressão simbólica.

Ambas as visões também apresentam a funcionalidade de filtrar os itens apresentados por meio de campos de busca localizados na parte superior. Inicialmente, apenas o filtro de nome é visível, mas a plataforma também implementa filtros adicionais que podem ser acessados por meio do menu de filtros avançados, localizado ao lado do campo de busca, os quais podem ser vistos na imagem 17.

As barras de navegação laterais tem como objetivo facilitar a navegação entre os principais elementos do sistema, permitindo que os usuários acessem espaços de trabalho e gerenciem datasets de forma eficiente. A área central implementa um sistema de abas para permitir que diferentes Profiles sejam abertas simultaneamente, promovendo a comparação e análise de resultados entre diferentes espaços de trabalho.

4.7.3.2 UPLOAD DE DATASETS

O upload de datasets é feito por meio da aba de upload, acessada na visão de datasets da barra de atividades. A figura 16 apresenta o estado inicial da tela, o qual espera que o usuário faça o upload de pelo menos um arquivo CSV. Após a escolha dos arquivos, a tela permite que o usuário configure o nome, descrição e papéis dos arquivos, os quais podem ser do tipo *Data* ou do tipo *Validation*. O primeiro é utilizado para o

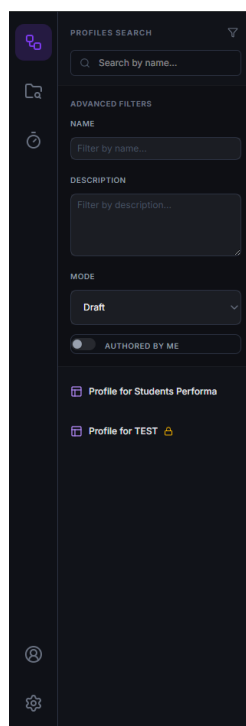


Figura 17 – Menu de filtros avançados presente nas visões de Profiles da plataforma Spectrum

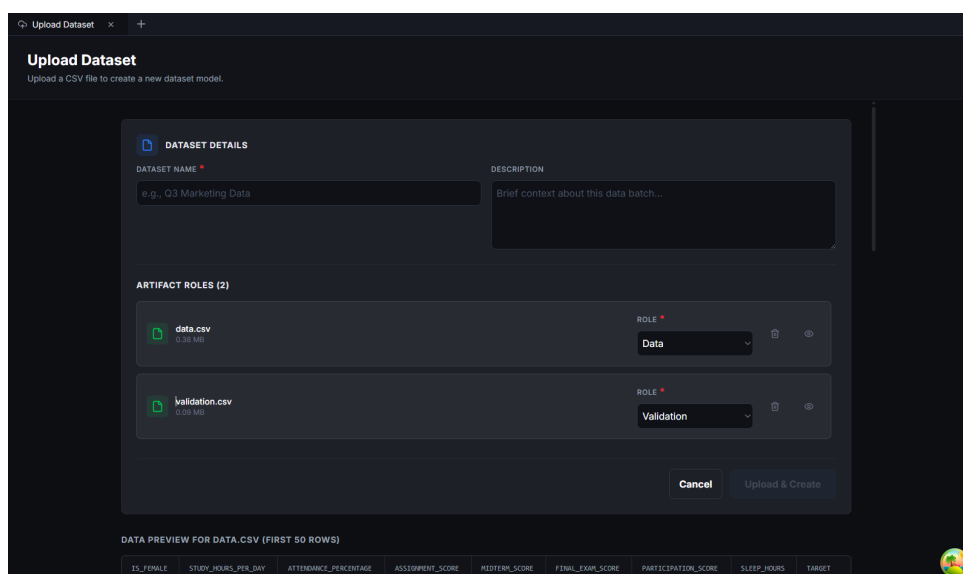
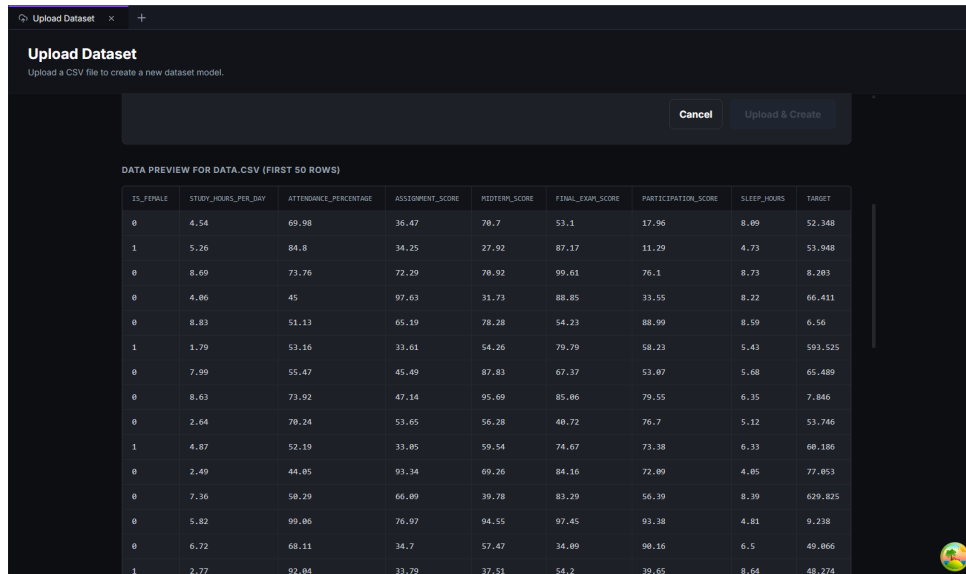


Figura 18 – Seção de configuração dos artefatos do dataset sendo criado na plataforma Spectrum

treinamento dos modelos de regressão simbólica, enquanto o segundo é utilizado para a validação dos modelos gerados. O upload de datasets é uma etapa fundamental para a utilização da plataforma, permitindo que os usuários forneçam os dados necessários para o treinamento e validação de seus modelos de regressão simbólica. As imagens 18 e 19 apresentam, respectivamente, a seção de configuração dos artefatos do dataset sendo criado, e a seção de pré-visualização dos dados presentes nos arquivos CSV enviados.



IS_FEMALE	STUDY_HOURS_PER_DAY	ATTENDANCE_PERCENTAGE	ASSIGNMENT_SCORE	MIDTERM_SCORE	FINAL_EXAM_SCORE	PARTICIPATION_SCORE	SLEEP_HOURS	TARGET
0	4.54	69.98	36.47	70.7	53.1	17.96	8.69	52.348
1	5.26	84.8	34.25	27.92	87.17	11.29	4.73	53.948
0	8.69	73.76	72.29	70.92	99.61	76.1	8.73	8.203
0	4.06	45	97.63	31.73	88.85	33.55	8.22	66.411
0	8.83	51.13	65.19	78.28	54.23	88.99	8.59	6.56
1	1.79	53.16	33.61	54.26	79.79	58.23	5.43	593.525
0	7.99	55.47	45.49	87.83	67.37	53.07	5.68	65.489
0	8.63	73.92	47.14	95.69	85.06	79.55	6.35	7.846
0	2.64	70.24	53.65	56.28	40.72	76.7	5.12	53.746
1	4.87	52.19	33.05	59.54	74.67	73.38	6.33	60.186
0	2.49	44.05	93.34	69.26	84.16	72.09	4.05	77.053
0	7.36	50.29	66.09	39.78	83.29	56.39	8.39	629.825
0	5.82	99.06	76.97	94.55	97.45	93.38	4.81	9.238
0	6.72	68.11	34.7	57.47	34.09	90.16	6.5	49.866
1	2.77	92.04	33.79	37.51	54.2	39.65	8.64	48.274

Figura 19 – Seção de pré-visualização dos dados presentes nos arquivos CSV enviados na plataforma Spectrum

Quando o botão de criar dataset é clicado, os arquivos e os metadados configurados são enviados para o back-end via requisição HTTP. Após a resposta do back-end, a plataforma redireciona o usuário para a visão de Datasets e invalida o cache do *React Query*, forçando uma nova requisição para o back-end para obter a lista atualizada de datasets, incluindo o novo dataset criado.

4.7.3.3 ABA DE PROFILES

A aba de Profiles é o espaço de trabalho central da plataforma, onde os usuários podem interagir com seus modelos de regressão e demais entidades do sistema. A parte superior da aba de Profiles contém seções para metadados (como nome e descrição), uma seção de datasets, que mostram quais datasets foram importados para a Profile, uma seção de Jobs, que mostram os Jobs configurados para a Profile, uma seção de modelos, e uma seção de blocos, que mostram os blocos de visualização presentes na Profile. A parte inferior da aba de Profiles é dedicada à área de blocos, onde os usuários podem configurar e organizar os blocos de visualização de acordo com suas necessidades específicas. As imagens 20 e 21 apresentam a tela de uma Profile, onde é possível observar as seções mencionadas.

Quando um bloco de consulta IQL é executado, o estado atual da Profile é atualizado no back-end. O front-end então envia uma requisição HTTP solicitando a execução da consulta IQL, a qual é processada por um *worker* de forma assíncrona. Durante a execução da consulta, os resultados parciais são enviados para o front-end por meio de Server Side Events (SSE), permitindo que os usuários visualizem em tempo real as etapas de execução de suas consultas. Ao final da execução, os resultados completos da consulta

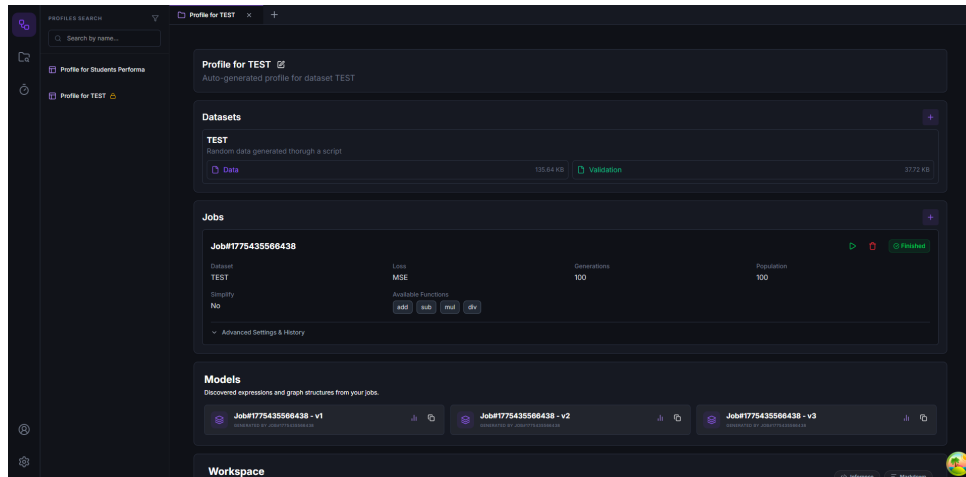


Figura 20 – Tela de uma Profile na plataforma Spectrum, onde é possível observar as seções de metadados, datasets, Jobs e blocos

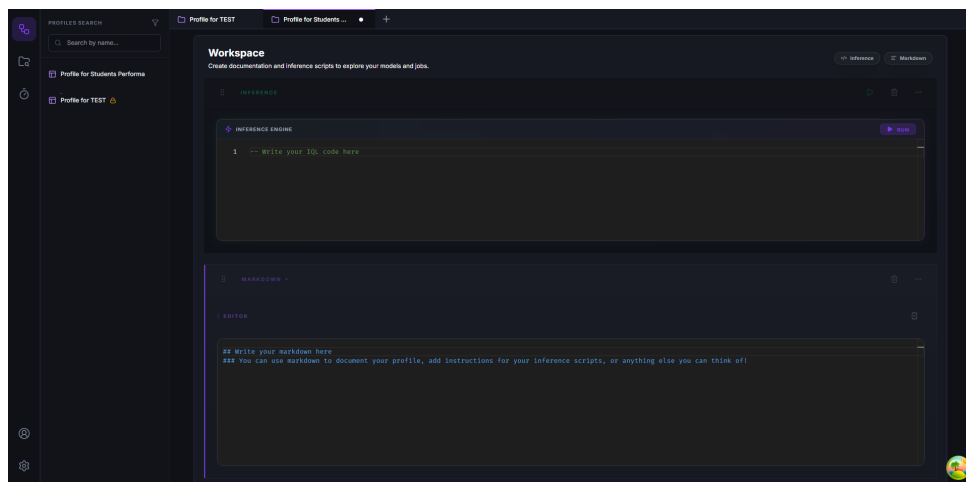


Figura 21 – Área de blocos de uma Profile na plataforma Spectrum, onde os usuários podem configurar e organizar os blocos de visualização de acordo com suas necessidades específicas

são armazenados no banco de dados como `InferenceResults`, permitindo que os usuários acessem e analisem os resultados de suas consultas IQL por meio da interface web da plataforma. A imagem 22 apresenta um exemplo de resultados de uma consulta IQL sendo exibidos em tempo real por meio de SSE.

Na seção de Jobs, os usuários podem configurar novos Jobs, visualizar Jobs configurados, executar Jobs, acompanhar o progresso de execução e analisar o histórico de execuções de cada Job. A figura 23 apresenta a seção de Jobs de uma Profile, onde é possível observar as funcionalidades mencionadas.

A figura 24 apresenta a tela de configuração de um novo Job, onde os usuários podem configurar os meta-parâmetros do algoritmo Eggp, como o número de gerações, o tamanho da população e as funções a serem utilizadas, entre outros. Após a configuração, o Job é criado e sua execução começa imediatamente, gerando um novo Run associado

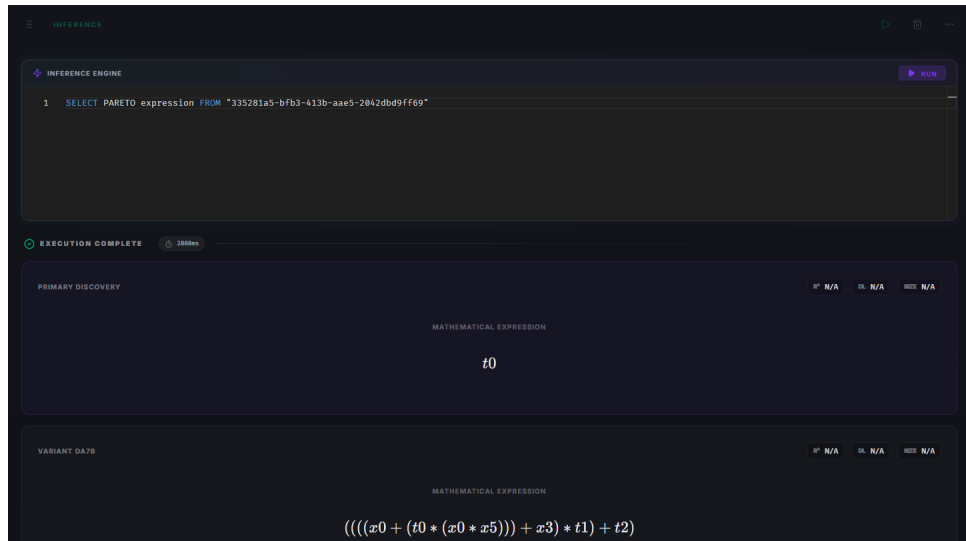


Figura 22 – Exemplo de resultados de uma consulta IQL sendo exibidos em tempo real por meio de Server Side Events (SSE) na plataforma Spectrum

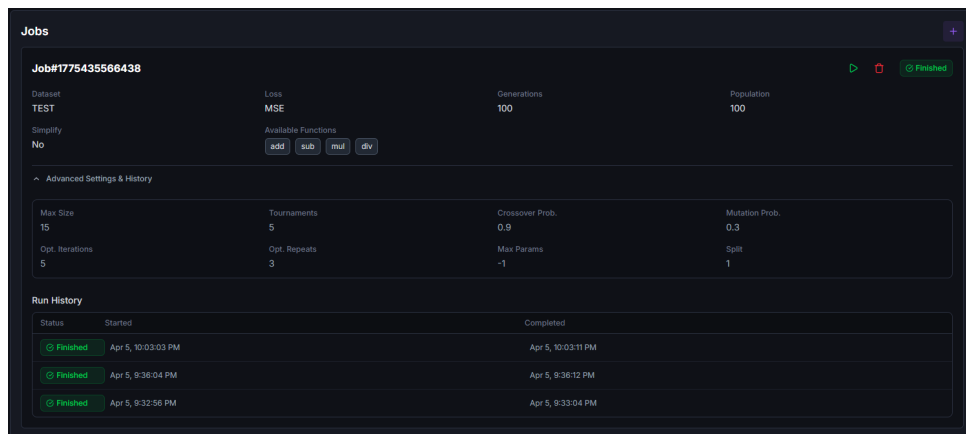


Figura 23 – Seção de Jobs de uma Profile na plataforma Spectrum, onde os usuários podem configurar novos Jobs, visualizar Jobs configurados, executar Jobs, acompanhar o progresso de execução e analisar o histórico de execuções de cada Job

ao Job. O progresso da execução do Job pode ser acompanhado em tempo real por meio da interface web, permitindo que os usuários monitorem o andamento do treinamento de seus modelos de regressão simbólica.

Na seção de modelos, os usuários podem visualizar os modelos de regressão simbólica gerados a partir de suas execuções, bem como acessar as métricas de performance associadas a cada modelo. A figura 25 apresenta a seção de modelos de uma Profile, onde é possível observar as funcionalidades mencionadas.

Ao clicar no botão de métricas, um modal é aberto exibindo a comparação entre as expressões da fronteira de pareto encontradas durante o processo de validação e a comparação entre os valores reais do conjunto de validação e os valores preditos por cada expressão, permitindo que os usuários analisem a performance de cada modelo de

New Job

Basic Settings

NAME * Job#1775942458409 DATASET * TEST

LOSS FUNCTION GENERATIONS POPULATION

MSE 100 100

AVAILABLE FUNCTIONS

Add x Sub x Mul x Div x

SIMPLIFY
APPLY ALGEBRAIC SIMPLIFICATION AFTER EVOLUTION ☐

Advanced Settings

MAX SIZE NUMBER OF TOURNAMENTS

15 5

CROSSOVER PROBABILITY MUTATION PROBABILITY

0.9 0.3

OPTIMIZATION ITERATIONS OPTIMIZATION REPEATS

5 3

MAX PARAM COUNT SPLIT

-1 1

Cancel Create Job

Figura 24 – Tela de configuração de um novo Job na plataforma Spectrum, onde os usuários podem configurar os meta-parâmetros do algoritmo Eggp, como o número de gerações, o tamanho da população e as funções a serem utilizadas, entre outros

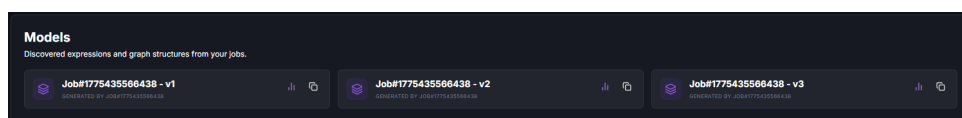


Figura 25 – Seção de modelos de uma Profile na plataforma Spectrum, onde é possível visualizar os modelos de regressão simbólica gerados a partir de suas execuções, bem como acessar as métricas de performance associadas a cada modelo

regressão simbólica gerado. A figura 26 apresenta a comparação entre as expressões da fronteira de pareto, enquanto a figura 27 apresenta a comparação entre os valores reais do conjunto de validação e os valores preditos por cada expressão.

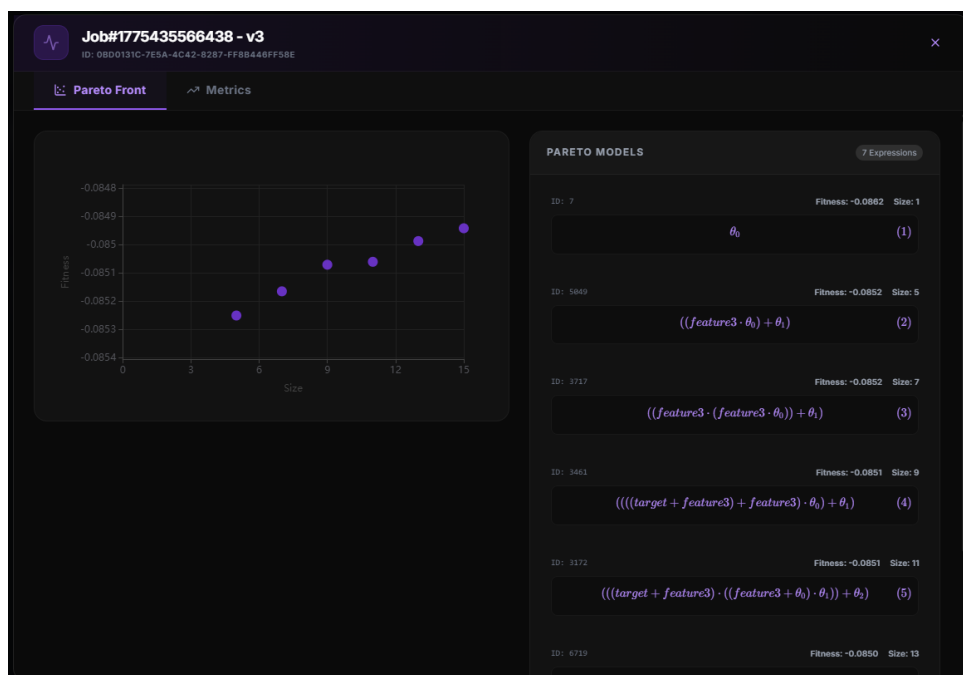


Figura 26 – Comparação entre as expressões da fronteira de pareto encontradas durante o processo de validação na plataforma Spectrum

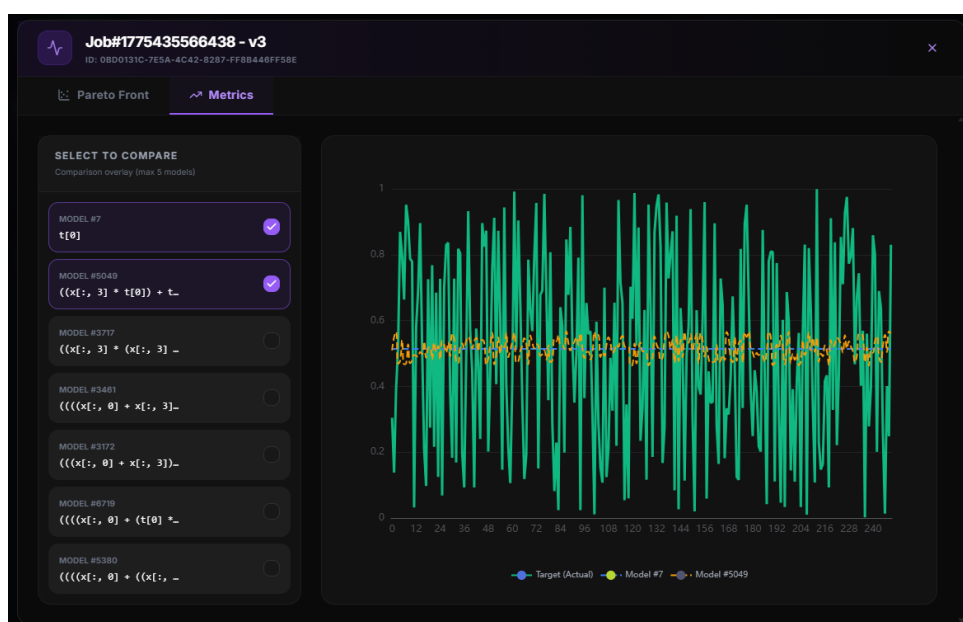


Figura 27 – Comparação entre os valores reais do conjunto de validação e os valores preditos por cada expressão na plataforma Spectrum

5 LINGUAGEM DE DOMÍNIO PARA EXPLORAÇÃO DE REGRESSÃO SIMBÓLICA

5.1 CONTEXTO

A plataforma proposta nesse projeto tem como um dos objetivos principais permitir a manipulação de modelos de regressão simbólica de maneira similar a outras ferramentas já bem conhecidas no mundo da análise de dados. Uma das ferramentas mais utilizadas na Ciência de Dados é a Structured Query Language (SQL), uma linguagem de domínio (SOMMERVILLE, 2011) que permite a interação com bancos de dados relacionais.

SQL é uma linguagem de domínio (Domain-Specific Language - DSL), ou seja, uma linguagem que busca resolver um problema específico do universo para qual ela foi pensada. No caso do SQL, o problema que ela resolve é a busca e processamento de dados em bancos de dados relacionais. Outras linguagens de domínio são amplamente utilizadas em computação para simplificar tarefas complexas por meio de abstrações declarativas.

Para facilitar o uso da ferramenta *rEGGression* (FRANÇA; KRONBERGER, 2025b) e permitir o uso da plataforma proposta por usuários que já têm conhecimento prático em outras ferramentas voltadas para predição de dados que se utilizam de dialetos SQL, além da plataforma, este trabalho propõe uma linguagem de domínio denominada Inference Query Language (IQL), com sintaxe similar ao SQL.

A Tabela 1 apresenta um quadro comparativo entre as estruturas tradicionais da linguagem SQL e os seus equivalentes desenvolvidos para a IQL. Enquanto o SQL opera sobre as tabelas e colunas relacionais, a IQL opera sobre o espaço de busca e extrai as propriedades abstratas das expressões matemáticas candidatas resultantes.

5.2 INFERENCE QUERY LANGUAGE

A Inference Query Language (IQL) foi concebida para permitir que usuários possam consultar soluções dentro de um modelo de regressão simbólica de forma declarativa. Em sistemas de regressão simbólica baseados em grafos de igualdade, como o Eggp (FRANÇA; KRONBERGER, 2025a), o espaço de soluções pode conter milhares de expressões equivalentes ou sub-expressões que podem ser de interesse para o pesquisador.

Tabela 1 – Comparação estrutural e comportamental entre estruturas primitivas da linguagem relacional SQL e a DSL Inference Query Language proposta

Estrutura	SQL (Banco de Dados)	IQL (Regressão Simbólica)
SELECT	Define as colunas a serem retornadas na projeção	Define as propriedades estruturais da equação (expression , size) projetadas
FROM	Especifica a tabela de origem no esquema relacional	Especifica o modelo para instanciar o subjacente <i>e-graph</i> em memória
WHERE	Filtra linhas de acordo com predicados	Filtra equações empíricas por complexidade ou taxa de erro
LIKE	Busca de similaridade léxica (%) sob dados em <i>strings</i>	Executa correspondência morfológica (<i>pattern matching</i>) na AST da equação
ORDER BY	Ordena a saída de modo indexado por colunas sequenciais	Ordena por métricas de aptidão intrínseca (<i>fitness</i>) ou complexidade (<i>size</i>)

Assim como o SQL, o IQL é uma linguagem declarativa. Linguagens declarativas são aquelas em que o programador descreve o que ele deseja obter, sem especificar o fluxo de controle ou os passos procedimentais para alcançar esse resultado (SOMMERVILLE, 2011). No contexto do IQL, o usuário pode especificar padrões matemáticos, restrições de complexidade ou métricas de erro, e o sistema é responsável por realizar a busca e filtragem de forma automática no *e-graph*.

5.2.1 GRAMÁTICA FORMAL

A IQL é composta por uma sintaxe inspirada no padrão ANSI SQL. A gramática formal da linguagem, apresentada na Figura 28, foi especificada em Extended Backus-Naur Form (EBNF) e define todas as produções expressamente aceitas pelo parser da plataforma.

A produção raiz *query* exige obrigatoriamente as cláusulas primárias **SELECT** e **FROM**. Todavia, as demais cláusulas de filtro operam de maneira inteiramente opcional. A linguagem opera em três sub-modos principais, controlados logicamente pelos modificadores que sucedem e interagem com a palavra-chave central **SELECT**:

- **TOP n** : Seleciona as n melhores expressões do *e-graph* de acordo com uma métrica de aptidão (*fitness*). Em caso de em que o modificador não é especificado, a linguagem considera **TOP 10** como valor nativo padrão para o ambiente de execução;
- **PARETO**: Seleciona as expressões que compõem a fronteira de Pareto do espaço de soluções, ou seja, aquelas que não são dominadas por nenhuma outra em termos

Figura 28 – Gramática formal da Inference Query Language (IQL) em notação EBNF, definindo a sintaxe completa para consultas declarativas sobre modelos de regressão simbólica baseados em e-graphs

<i>query</i>	::=	SELECT [<i>modifier</i>] [DISTRIBUTION] <i>column-list</i> FROM <i>identifier</i> [<i>where-clause</i>] [<i>pattern-clause</i>] [<i>order-clause</i>] [<i>at-least-clause</i>] [<i>limit-clause</i>]
<i>modifier</i>	::=	TOP <i>integer</i> PARETO
<i>column-list</i>	::=	<i>column</i> (, <i>column</i>)*
<i>column</i>	::=	<i>identifier</i> <i>predict-call</i>
<i>predict-call</i>	::=	PREDICT (<i>mapping</i> (, <i>mapping</i>)*)
<i>mapping</i>	::=	<i>identifier</i> = <i>number</i>
<i>where-clause</i>	::=	WHERE <i>condition</i> (<i>bool-op</i> <i>condition</i>)*
<i>condition</i>	::=	<i>identifier</i> <i>comparator</i> <i>number</i>
<i>comparator</i>	::=	= < > <= >=
<i>bool-op</i>	::=	AND OR
<i>pattern-clause</i>	::=	PATTERN IS LIKE <i>pattern-expr</i>
<i>pattern-expr</i>	::=	<i>identifier</i> <i>pattern-expr</i> <i>arith-op</i> <i>pattern-expr</i>
<i>arith-op</i>	::=	+ - * /
<i>order-clause</i>	::=	ORDER BY <i>identifier</i>
<i>at-least-clause</i>	::=	AT LEAST <i>integer</i>
<i>limit-clause</i>	::=	LIMIT <i>integer</i>
<i>identifier</i>	::=	<i>letter</i> (<i>letter</i> <i>digit</i> <i>_</i>)*
<i>number</i>	::=	<i>digit</i> + [. <i>digit</i> +]
<i>integer</i>	::=	<i>digit</i> +

de aptidão e complexidade. A fronteira de Pareto é um conceito fundamental em otimização multiobjetivo, onde se busca encontrar soluções que sejam ótimas em múltiplos critérios simultaneamente;

- **DISTRIBUTION**: Em vez de retornar as expressões diretamente, este modificador instrui a linguagem a calcular a distribuição de um determinado padrão de expressão dentro do *e-graph*, retornando a frequência de ocorrência e a aptidão média para cada padrão encontrado. Este modo é especialmente útil para análises estatísticas e para entender a prevalência de certos tipos de expressões dentro do espaço de soluções.

Independentemente do modo de execução, a IQL opera sobre um modelo específico de regressão simbólica, especificado na cláusula **FROM**. Antes da execução da consulta, a plataforma fornece ao ambiente de execução uma lista de modelos disponíveis na **profile**

associada ao código IQL. A cláusula *FROM* deve referenciar um desses modelos, via nome ou identificador único, para que a consulta seja processada. O modelo referenciado é então instanciado em memória, e o *e-graph* correspondente é construído para que as operações de busca e filtragem possam ser realizadas.

Diferentemente do SQL, onde a cláusula **SELECT** é seguida por uma lista de colunas a serem recuperadas da tabela especificada na cláusula **FROM**, na IQL a cláusula **SELECT** é seguida por uma lista de propriedades ou métricas das expressões matemáticas que se deseja obter como resultado da consulta. Nesse sentido, todas as consultas operam sobre o mesmo espaço de propriedades selecionáveis, sendo elas:

- **id**: O identificador único da expressão dentro do *e-graph*, que pode ser utilizado para referência em outras partes da consulta ou para operações de filtragem;
- **expression**: A expressão matemática completa da solução encontrada, representada como uma string ou uma estrutura de dados que pode ser interpretada posteriormente;
- **dl**: Comprimento de descrição da expressão, que é uma representação da quantidade de informação necessária para descrever a expressão, geralmente relacionada à sua complexidade estrutural;
- **fitness**: A métrica de aptidão associada à expressão;
- **latex**: A representação em LaTeX da expressão, que pode ser útil para visualização ou documentação;
- **numpy**: A representação da expressão em formato compatível com a biblioteca NumPy, que pode ser utilizada para avaliação numérica ou manipulação programática;
- **parameters**: Uma lista de parâmetros livres presentes na expressão;
- **size**: A métrica de complexidade associada à expressão;
- **pattern**: A estrutura de padrão da expressão, que pode ser utilizada para correspondência de padrões ou análises morfológicas;
- **frequency**: A frequência de ocorrência da expressão dentro do *e-graph*, que é especialmente relevante quando o modificador **DISTRIBUTION** é utilizado.

Além disso, também é possível utilizar funções pré-definidas dentro da lista de seleção. A função mais relevante é a função de predição **PREDICT**, que permite ao usuário realizar predições com base em uma expressão específica, fornecendo um mapeamento de variáveis para valores numéricos. Por exemplo, a chamada **PREDICT(x0 = 1.5, x1 =**

2.0) instrui o sistema a avaliar a expressão selecionada substituindo as variáveis **x0** e **x1** pelos valores 1.5 e 2.0, respectivamente, e retornar o resultado da avaliação.

A cláusula **WHERE** aplica filtros condicionais sobre as expressões candidatas, utilizando comparadores tradicionais como **=**, **<**, **>** e seus equivalentes de comparação. Esses filtros permitem restringir o conjunto de soluções retornadas com base em métricas como complexidade, número de parâmetros livres ou custo da expressão. Além disso, é possível filtrar especificamente pelos identificadores únicos das expressões, o que é útil para operações de referência cruzada ou para análises específicas sobre soluções previamente identificadas, desde que conhecidos esses identificadores. Múltiplas condições podem ser combinadas utilizando os operadores lógicos **AND** e **OR**, permitindo a construção de filtros complexos e específicos para as necessidades do usuário.

A cláusula **PATTERN IS LIKE** realiza uma filtragem baseada nos padrões matemáticos contidos nas expressões geradas pelo modelo. Essa cláusula é especialmente útil para identificar expressões que compartilham uma estrutura comum, independentemente dos parâmetros específicos ou da complexidade. Por exemplo, a consulta **PATTERN IS LIKE sin(v0)** retornaria todas as expressões que contêm a função seno aplicada a uma variável, independentemente de outras operações ou termos presentes na expressão. A correspondência de padrões é realizada através da análise da Árvore de Sintaxe Abstrata (AST) das expressões, possibilitada pela estrutura de dados do *e-graph* que mantém a representação abstrata da expressão matemática. Isso permite uma correspondência morfológica eficiente e flexível, que pode ser utilizada para explorar o espaço de soluções de maneira mais direcionada e intuitiva.

A cláusula **ORDER BY** permite a ordenação dos resultados com base em uma métrica específica, como aptidão ou complexidade. Isso é especialmente útil para priorizar as soluções mais relevantes ou mais simples, dependendo do contexto da análise. A ordenação é realizada de forma ascendente por padrão, mas pode ser configurada para ordenação descendente se necessário.

A cláusula **AT LEAST** é utilizada para especificar um número mínimo de ocorrências de um determinado padrão ou expressão dentro do *e-graph*. Isso é particularmente útil quando se deseja identificar expressões que são recorrentes ou que possuem uma frequência significativa dentro do espaço de soluções, o que pode indicar uma relevância maior ou uma tendência estrutural importante. Já a cláusula **LIMIT** restringe o número total de resultados retornados pela consulta, o que é útil para evitar sobrecarga de informações e para focar em um subconjunto mais gerenciável de soluções, especialmente quando o espaço de soluções é muito grande.

5.2.2 EXEMPLOS DE CONSULTAS

Abaixo são apresentados exemplos de como a IQL pode ser utilizada para extrair informações específicas de um modelo de regressão simbólica:

1. **Busca por modelos simples na fronteira de Pareto:**

```
SELECT PARETO expression , size FROM my_model WHERE size  
    <= 5
```

2. **Identificação de expressões que contenham um padrão específico, nesse caso, a função seno aplicada a uma variável:**

```
SELECT TOP 10 expression , fitness , size FROM my_model  
PATTERN IS LIKE sin(v0)
```

3. **Busca por expressões que contenham um padrão de multiplicação entre duas variáveis:**

```
SELECT PARETO expression , size FROM my_model  
PATTERN IS LIKE x1 * x2
```

4. **Análise de distribuição de padrões específicos dentro do espaço de soluções, buscando identificar quais padrões são mais frequentes e qual é a aptidão média associada a esses padrões:**

```
SELECT TOP 100 DISTRIBUTION pattern , frequency , fitness  
FROM my_model AT LEAST 5 LIMIT 500
```

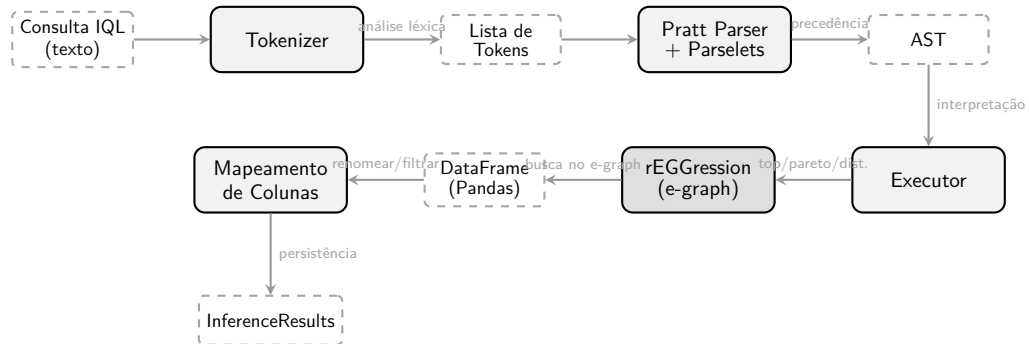
5. **Realização de predições com base em expressões selecionadas, substituindo variáveis por valores numéricos específicos para avaliar o resultado da expressão sob essas condições:**

```
SELECT TOP 5 expression , fitness , PREDICT(x0 = 1.5 , x1 =  
    2.0)  
FROM my_model WHERE size <= 10
```

5.3 EXECUÇÃO

A arquitetura de execução da IQL é composta por um *pipeline* de processamento que envolve várias etapas, desde a análise léxica e sintática da consulta até a execução propriamente dita e o retorno dos resultados. A Figura 29 ilustra esse processo de maneira detalhada, mostrando como a consulta textual é transformada em uma estrutura de dados que pode ser interpretada e executada para extrair as informações desejadas do modelo de regressão simbólica.

Figura 29 – Pipeline de processamento, análise e execução de uma consulta IQL, desde a entrada textual oriunda da *request* do usuário até o armazenamento dos dados



5.3.1 ANÁLISE LÉXICA (TOKENIZAÇÃO)

O processo de análise léxica, ou tokenização, é a primeira etapa do *pipeline* de execução da IQL. Nessa etapa, a consulta textual fornecida pelo usuário é processada para identificar os componentes léxicos que compõem a consulta, como palavras-chave, identificadores, operadores e literais. O resultado dessa etapa é uma lista de tokens, onde cada token representa um componente específico da consulta. O tokenizer é responsável por reconhecer a sintaxe da linguagem, identificar os tipos de tokens e lidar com possíveis erros de formatação ou caracteres inválidos. Por exemplo, para a consulta:

SELECT TOP 10 expression , fitness **FROM** model **WHERE** fitness < 1.5

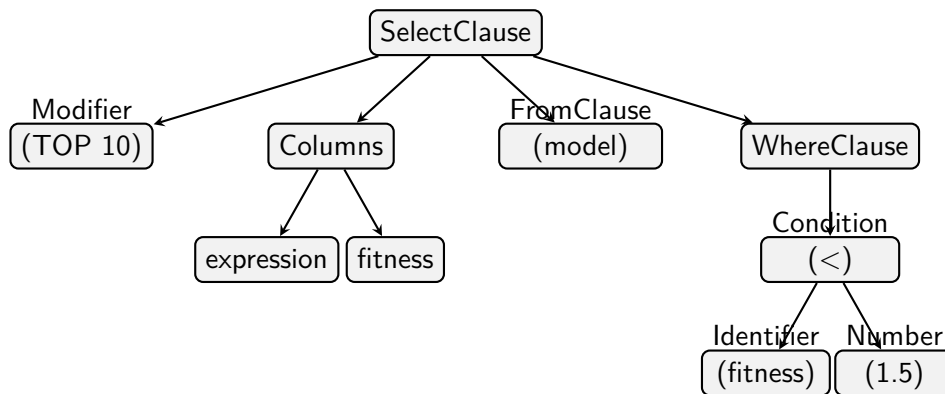
O tokenizer processaria essa string e geraria uma lista de tokens como a seguinte:

```
[
  Token(SELECT, "SELECT"),
  Token(TOP, "TOP"),
  Token(NUMBER, "10"),
  Token(IDENTIFIER, "expression"),
  Token(COMMA, ","),
  Token(IDENTIFIER, "fitness"),
  Token(FROM, "FROM"),
  Token(IDENTIFIER, "model"),
  Token(WHERE, "WHERE"),
  Token(IDENTIFIER, "fitness"),
  Token(OPERATOR, "<"),
  Token(NUMBER, "1.5")
]
```

5.3.2 ANÁLISE SINTÁTICA E O PRATT PARSER

A segunda fase do *pipeline* de execução da IQL é a análise sintática. Nesta etapa, a lista de tokens gerada anteriormente é processada para construir uma árvore que representa a estrutura hierárquica da consulta, conhecida como Árvore de Sintaxe Abstrata (AST). A análise sintática é realizada utilizando um Pratt Parser, que é uma técnica de parsing eficiente e flexível para linguagens com operadores de precedência (PRATT, 1973). O Pratt Parser utiliza uma abordagem baseada em *parselets*, onde cada tipo de token tem um *parselet* associado que define como ele deve ser processado em relação aos outros tokens. O resultado dessa etapa é uma AST que representa a estrutura lógica da consulta, onde cada nó da árvore corresponde a um componente da consulta, como cláusulas, expressões ou operadores. Por exemplo, para a consulta mencionada anteriormente, a AST resultante poderia ser representada graficamente como mostrado na Figura 30, onde os nós representam as diferentes partes da consulta e as arestas representam as relações hierárquicas entre elas. Essa estrutura é fundamental para a etapa de execução, pois permite que o sistema interprete a consulta de maneira eficiente e extraia as informações desejadas do modelo de regressão simbólica.

Figura 30 – Visualização da Árvore de Sintaxe Abstrata (AST) do Pratt Parser



5.3.3 EXECUÇÃO

A etapa de execução é onde a consulta, representada pela AST gerada na etapa anterior, é interpretada e processada para extrair as informações desejadas do modelo de regressão simbólica. O executor é responsável por percorrer a AST, identificar as operações a serem realizadas e interagir com o modelo de regressão simbólica (rEGGression) para obter os resultados. Dependendo dos modificadores e cláusulas presentes na consulta, o executor pode realizar diferentes tipos de buscas no *e-graph*, como buscar as melhores expressões de acordo com uma métrica de aptidão, identificar expressões que correspondam a um padrão específico ou calcular distribuições de padrões dentro do espaço de soluções. O resultado da execução é então formatado em um DataFrame do Pandas, que é uma estrutura de dados tabular amplamente utilizada em Python para manipulação e

análise de dados. Esse DataFrame é posteriormente mapeado para um formato específico de resultados (InferenceResults) que pode ser armazenado ou retornado ao usuário para visualização ou análise adicional.

6 CONCLUSÃO

6.1 LIMITAÇÕES E TRABALHOS FUTUROS

A plataforma apresentada neste trabalho, embora apresente funcionalidades comparáveis às ferramentas tradicionais de regressão simbólica, ainda possui algumas limitações e oportunidades de trabalho futuro. Dado que a plataforma proposta visa ser um ambiente de trabalho integrado que permita o gerenciamento de pipelines de dados que dependam de modelos de regressão simbólica, a falta de capacidades de agendamento de treinamentos ou de inferência de dados em tempo real é uma limitação importante a ser abordada. A plataforma contém, atualmente, listas fixas de funções de perda e de não terminais, o que pode limitar a flexibilidade dos usuários na definição de seus próprios problemas de regressão simbólica. A implementação de uma interface que permita aos usuários definirem suas próprias funções de perda e conjuntos de não terminais, bem como a capacidade de importar funções personalizadas, seria uma melhoria significativa para a plataforma, aumentando sua versatilidade e aplicabilidade a uma gama mais ampla de problemas.

Além disso, a atual implementação do modelo de *workers* não é resistente a falhas, o que pode levar a perda de dados ou interrupção de processos em caso de falhas no sistema. A utilização de um padrão de outbox para garantir a entrega de mensagens e a implementação de mecanismos de retry para lidar com falhas temporárias são melhorias que podem ser implementadas para aumentar a robustez do sistema, e as quais já são possíveis de serem implementadas via o conceito de recursos transacionais e do RabbitMQ.

A linguagem de consulta IQL também pode ser expandida para incluir funcionalidades adicionais que permitem consultas mais complexas e específicas, como a capacidade de realizar junções entre diferentes modelos ou a introdução de funções agregadas para análise estatística dos resultados. A implementação de um otimizador de consultas que possa reescrever consultas para melhorar a eficiência da execução também é uma área promissora para trabalho futuro. Atualmente, a linguagem IQL também não suporta consultas aninhadas, o que limita o poder expressivo da linguagem. A introdução de subconsultas e a capacidade de realizar consultas recursivas são melhorias que podem ser implementadas para aumentar a flexibilidade da linguagem.

Para além dessas melhorias, a plataforma pode ser ampliada para suportar outros algoritmos de regressão simbólica, os quais podem ser utilizados no rEGGression, como o PySR ou o Operon. A integração de múltiplos algoritmos de regressão simbólica permitiria aos usuários comparar os resultados obtidos por diferentes abordagens e escolher a mais

adequada para o seu problema específico, aumentando a versatilidade da plataforma. Por fim, a expansão das funções disponíveis para o treinamento dos modelos como a adição de funções matriciais podem permitir a aplicação da plataforma em uma gama mais ampla de problemas, incluindo aqueles que envolvem dados multivariados, séries temporais ou componentes não numéricas como texto ou imagens que podem ser processados por meio de técnicas de embedding e posteriormente utilizados como variáveis de entrada para os modelos de regressão simbólica.

6.2 CONCLUSÃO

Este trabalho apresentou o Spectrum, uma plataforma *web* moderna e extensível para a manipulação de modelos de regressão simbólica. Através da implementação de uma arquitetura baseada em monólito modular e do uso de tecnologias como FastAPI, React e RabbitMQ, foi possível criar um ambiente robusto que se equipara às ferramentas tradicionais, mas com a vantagem de oferecer execução remota e um ambiente de trabalho integrado.

Além disso, o desenvolvimento da Inference Query Language (IQL) permitiu que os usuários possam interagir com os modelos de regressão simbólica de maneira declarativa, facilitando a extração de conhecimento e a aplicação de filtros baseados em métricas e padrões estruturais. A IQL é uma linguagem de domínio inspirada no SQL, que se integra ao back-end da plataforma para realizar consultas eficientes sobre o espaço de soluções dos modelos. O uso da IQL permite que o usuário aplique o conhecimento prévio sobre o fenômeno físico diretamente no processo de seleção de modelos, algo que é dificultado em ferramentas de regressão tradicionais que retornam apenas a melhor solução numérica disponível.

De modo geral, o Spectrum apresenta uma solução moderna e flexível para a manipulação de modelos de regressão simbólica, oferecendo uma experiência de usuário aprimorada e a capacidade de realizar consultas complexas sobre os modelos. Com as melhorias propostas para o futuro, a plataforma tem o potencial de se tornar uma ferramenta ainda mais poderosa e indispensável para pesquisadores e profissionais que trabalham com regressão simbólica.

REFERÊNCIAS

ANGELIS, Dimitrios; SOFOS, Filippas; KARAKASIDIS, Theodoros E. Artificial Intelligence in Physical Sciences: Symbolic Regression Trends and Perspectives. **Archives of Computational Methods in Engineering**, v. 30, n. 6, p. 3845–3865, jun. 2023. ISSN 1886-1784. DOI: [10.1007/s11831-023-09922-z](https://doi.org/10.1007/s11831-023-09922-z). Disponível em: [10.1007/s11831-023-09922-z](https://doi.org/10.1007/s11831-023-09922-z).

BARTLETT, Deaglan J.; DESMOND, Harry; FERREIRA, Pedro G. Exhaustive Symbolic Regression. **IEEE Transactions on Evolutionary Computation**, Institute of Electrical and Electronics Engineers (IEEE), v. 28, n. 4, p. 950–964, ago. 2024. ISSN 1941-0026. DOI: [10.1109/tevc.2023.3280250](https://doi.org/10.1109/tevc.2023.3280250). Disponível em: <http://dx.doi.org/10.1109/TEVC.2023.3280250>.

BEYER, Hans-Georg; SCHWEFEL, Hans-Paul. Evolution strategies – A comprehensive introduction. **Natural Computing**, v. 1, n. 1, p. 3–52, mar. 2002. ISSN 1572-9796. DOI: [10.1023/A:1015059928466](https://doi.org/10.1023/A:1015059928466). Disponível em: <https://doi.org/10.1023/A:1015059928466>.

COHN, Mike. **User stories applied: For agile software development**. [S.l.]: Addison-Wesley Professional, 2004.

EVANS, Eric. **Domain-driven design: tackling complexity in the heart of software**. [S.l.]: Addison-Wesley Professional, 2004.

FRANÇA, Fabricio Olivetti de; KRONBERGER, Gabriel. Improving Genetic Programming for Symbolic Regression with Equality Graphs. In: PROCEEDINGS of the Genetic and Evolutionary Computation Conference. Malaga, Spain: Association for Computing Machinery, 2025. (GECCO '25). ISBN 9798400714658. DOI: [10.1145/3712256.3726383](https://doi.org/10.1145/3712256.3726383). arXiv: [2501.17848 \[cs.LG\]](https://arxiv.org/abs/2501.17848). Disponível em: <https://doi.org/10.1145/3712256.3726383>.

_____. rEGGression: an Interactive and Agnostic Tool for the Exploration of Symbolic Regression Models. In: PROCEEDINGS of the Genetic and Evolutionary Computation Conference. Malaga, Spain: Association for Computing Machinery, 2025. (GECCO '25). ISBN 9798400714658. DOI: [10.1145/3712256.3726385](https://doi.org/10.1145/3712256.3726385). arXiv: [2501.17859 \[cs.LG\]](https://arxiv.org/abs/2501.17859). Disponível em: <https://doi.org/10.1145/3712256.3726385>.

HEURISTICLAB TEAM. **Documentation/About HeuristicLab; HeuristicLab — dev.heuristiclab.com**. [S.l.: s.n.], 2013. <https://dev.heuristiclab.com/trac.fcgi/wiki/Documentation/AboutHeuristicLab>. [Accessed 19-08-2025].

- IMAI ALDEIA, Guilherme Seidyo; FRANÇA, Fabrício Olivetti de. Lightweight Symbolic Regression with the Interaction - Transformation Representation. In: 2018 IEEE Congress on Evolutionary Computation (CEC). [S.l.: s.n.], 2018. P. 1–8. DOI: [10.1109/CEC.2018.8477951](https://doi.org/10.1109/CEC.2018.8477951).
- KATOCH, Sourabh; CHAUHAN, Sumit Singh; KUMAR, Vijay. A review on genetic algorithm: past, present, and future. **Multimedia Tools and Applications**, v. 80, n. 5, p. 8091–8126, fev. 2021. ISSN 1573-7721. DOI: [10.1007/s11042-020-10139-6](https://doi.org/10.1007/s11042-020-10139-6). Disponível em: <https://doi.org/10.1007/s11042-020-10139-6>.
- KRONBERGER, Gabriel; BURLACU, Bogdan et al. **Symbolic Regression**. [S.l.: s.n.], jul. 2024. ISBN 9781315166407. DOI: [10.1201/9781315166407](https://doi.org/10.1201/9781315166407).
- KRONBERGER, Gabriel; OLIVETTI DE FRANÇA, Fabricio et al. The Inefficiency of Genetic Programming for Symbolic Regression. In: _____. **Parallel Problem Solving from Nature – PPSN XVIII**. Cham: Springer Nature Switzerland, 2024. P. 273–289. ISBN 978-3-031-70055-2.
- LAMBORA, Annu; GUPTA, Kunal; CHOPRA, Kriti. Genetic Algorithm- A Literature Review. In: 2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon). [S.l.: s.n.], 2019. P. 380–384. DOI: [10.1109/COMITCon.2019.8862255](https://doi.org/10.1109/COMITCon.2019.8862255).
- MAKKE, Nour; CHAWLA, Sanjay. Interpretable scientific discovery with symbolic regression: a review. **Artificial Intelligence Review**, v. 57, n. 1, p. 2, jan. 2024. ISSN 1573-7462. DOI: [10.1007/s10462-023-10622-0](https://doi.org/10.1007/s10462-023-10622-0). Disponível em: <https://doi.org/10.1007/s10462-023-10622-0>.
- MARTIN, Robert C. **Clean Code: A Handbook of Agile Software Craftsmanship**. Upper Saddle River, NJ: Prentice Hall, 2008. ISBN 978-0132350884.
- OLIVETTI DE FRANÇA, Fabrício. A greedy search tree heuristic for symbolic regression. **Information Sciences**, Elsevier BV, 442–443, p. 18–32, mai. 2018. ISSN 0020-0255. DOI: [10.1016/j.ins.2018.02.040](https://doi.org/10.1016/j.ins.2018.02.040). Disponível em: <http://dx.doi.org/10.1016/j.ins.2018.02.040>.
- PRATT, Vaughan R. Top down operator precedence. In: PROCEEDINGS of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages. [S.l.: s.n.], 1973. P. 41–51.
- SOMMERVILLE, Ian. **Software Engineering**. 9th. [S.l.]: Pearson Education, 2011.
- STULP, Freek; SIGAUD, Olivier. Many regression algorithms, one unified model: A review. **Neural Networks**, v. 69, p. 60–79, 2015. ISSN 0893-6080. DOI: <https://doi.org/10.1016/j.neunet.2015.05.005>. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0893608015001185>.

TURINGBOT SOFTWARE. **TuringBot: Symbolic Regression Software.**

[S.l.: s.n.], 2020. <https://turingbotsoftware.com>. [Accessed 23-03-2026].

WAGNER, Stefan et al. Architecture and Design of the HeuristicLab Optimization Environment. In: ADVANCED Methods and Applications in Computational Intelligence. [S.l.]: Springer, 2014. (Topics in Intelligent Engineering and Informatics). P. 197–261. DOI: [10.1007/978-3-319-01433-3_8](https://doi.org/10.1007/978-3-319-01433-3_8).